



UPPSALA
UNIVERSITET

U.U.D.M. Project Report 2026:22

Examensarbete C, 15 hp
Kandidatprogrammet i matematik

Juni 2026

Comparative Analysis of Metaheuristics for Constructing D-Optimal Designs in Non-Linear Dose-Response Models

Simon Andebrhan Habtesion

Matematiska institutionen, Uppsala universitet

Handledare: Yevgen Ryeznic

Examinator: Martin Herschend

Contents

1	Introduction	3
2	Background	4
3	Methods and Theory	6
3.1	D-Optimality	6
3.1.1	Model and Design Framework	6
3.1.2	Fisher Information	8
3.1.3	D-Optimal Design	11
3.1.4	The General Equivalence Theorem	13
3.2	Models	15
3.2.1	Michaelis-Menten Model	16
3.2.2	E _{max} Model	18
3.3	Metaheuristics	20
3.3.1	Particle Swarm Optimization	20
3.3.2	Grey Wolf Optimizer	22
4	Approach	26
4.1	Use of Artificial Intelligence	26
4.2	Setup and Workflow	27
4.3	Implementation of PSO and GWO for D-Optimal Designs	36
4.3.1	Implementation Procedure	36
4.3.2	Settings for Computing Designs	40
4.4	Optimality Check	40
4.4.1	Implementation Procedure	40
4.4.2	Settings for Optimality Check	42
4.5	PSO vs. GWO	42
4.5.1	Implementation Procedure	42

4.6	Response Simulation and Estimation of Parameters	44
4.6.1	Implementation Procedure	44
4.6.2	Settings for Simulations	45
5	Results	47
5.1	Computation of Candidate D-Optimal Designs	47
5.2	Visualization and Optimality Check	48
5.3	PSO vs. GWO	55
5.4	Response Simulation and Estimation of Parameters	60
6	Discussion	70
6.1	Obtained Designs and Local D-Optimality	70
6.2	Comparison Between PSO and GWO	71
6.3	Response Simulation and Parameter Simulation	73
6.4	Conclusion	74
7	Appendix	76
7.1	Code Source Repository	76

Chapter 1

Introduction

Optimal design theory is an important area of statistics concerned with selecting experimental conditions that provide as much information as possible about the unknown parameters in a regression model. Two commonly used nonlinear models are the Michaelis-Menten model and the general Emax model, which often arise in fields such as biochemistry, pharmacology, and medicine, where precise parameter estimation is essential.

An issue in nonlinear optimal design is that the Fisher Information, which is used to quantify how informative the data is about the parameter, depends on the unknown parameter vector itself. Consequently, optimal designs typically depend on assumed parameter values, leading to the concept of locally optimal designs. In many cases, analytical solutions are difficult or impossible to derive, and numerical optimization methods must therefore be used.

This thesis considers locally D-optimal designs and metaheuristic optimization algorithms for solving the corresponding optimization problem. More specifically, this study investigates the use of Particle Swarm Optimization (PSO) and the Grey Wolf Optimizer (GWO) for constructing locally D-optimal designs for the Michaelis-Menten and Emax models.

The resulting designs are evaluated using the General Equivalence Theorem and, through simulations, the precision of the parameter estimation is studied. The main purpose is therefore both to look into the practicality of the metaheuristics in nonlinear optimal design and to compare their performance.

Chapter 2

Background

Finding D-optimal designs requires solving optimization problems over the design space that can in many cases be challenging, especially for nonlinear models, where the objective function may not be differentiable and can contain multiple local optima. Even though classical optimization methods can perform well in linear cases, they may fail or converge to local optima in nonlinear problems.¹

To address methods for these challenges, this thesis considers **metaheuristic optimization algorithms**. Metaheuristics are stochastic, general purpose optimization methods that do not require technical assumptions to work well and are capable of exploring complex search spaces effectively.²

They normally combine **exploration** (searching new solutions) and **exploitation** (refining known good solutions), and are particularly useful when:

- the objective function is difficult to differentiate, and
- global optimality is difficult to guarantee using classical methods.

In such contexts, several studies have considered optimal designs for dose-response models, such as the 2-parameter Michaelis-Menten models and the 3-parameter Emax model. In particular, P.-Y. Chen, R.-B. Chen, Tung, et

¹Ping-Yang Chen, Ray-Bing Chen, and Weng Kee Wong (2022). “Particle swarm optimization for searching efficient experimental designs: A review”. In: *WIREs Computational Statistics*, e1578. DOI: 10.1002/wics.1578

²P.-Y. Chen, R.-B. Chen, and Wong 2022.

al. (2017) studied standardized maximin D-optimal designs for the estimation of unknown parameters in enzyme kinetic models, solving a complex optimization problem using Particle Swarm Optimization.

This thesis follows steps motivated by their work, but takes, however, a different approach. Rather than focusing on standardized maximin optimality under parameter uncertainty, this study considers locally D-optimal designs for fixed assumed parameter values. This allows for a more direct study of the optimization problem while also providing a framework for comparing metaheuristic optimization algorithms in the context of D-optimal designs for non-linear models.

Chapter 3

Methods and Theory

3.1 D-Optimality

This section establishes the model and design framework, includes theory regarding Fisher information, and introduces D-optimality and the General Equivalence Theorem (GET). The objective function of interest and the concept of local D-optimality are also introduced here.

3.1.1 Model and Design Framework

In the context of experimental design, the main objective is to select experimental conditions that yield the most informative data for estimating model parameters. Let $\Theta \subset \mathbb{R}^p$ be the set of all plausible values that the p -dimensional parameter $\boldsymbol{\theta}$ can take and let $\mathcal{X}$ be the space of all design points. Consider the following general regression model:

$$y = \eta(x, \boldsymbol{\theta}) + \epsilon \quad (3.1)$$

where $x \in \mathcal{X}$, $\boldsymbol{\theta} \in \Theta$ and $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is a random error term. Then, y is the **response** generated according to (3.1).¹ For this particular framework, σ^2 is assumed to be known.

¹Valerii V. Fedorov and Peter Hackl (2025). *Model-Oriented Design of Experiments*. 2nd ed. Vol. 196. Lecture Notes in Statistics. Springer. ISBN: 978-1-0716-4301-3. DOI: 10.1007/978-1-0716-4302-0

The design space $\mathcal{X}$ represents all valid experimental conditions and the experimenter's task is to select values of x from $\mathcal{X}$ that are informative about the parameter in order to estimate θ precisely.¹

An **experimental design** is defined as a discrete probability measure:

$$\xi = \{(x_i, w_i)\}_{i=1}^k \quad (3.2)$$

where:

- $x_i \in \mathcal{X}$ are the **support points**: x -values at which the data is observed;
- $w_i \geq 0$ are **weights** satisfying: $\sum_{i=1}^k w_i = 1$.

Each weight w_i represents the proportion of measurements taken at x_i and the support points are the x -values from the design space for which $w_i > 0$, $i = 1, 2, \dots, k$. The **support of a design** is the set of all support points and is denoted S_ξ . Thus, if n observations are collected, approximately $n_i = n \cdot w_i$ observations are taken at point x_i .² In other words, a design specifies how observations are to be collected across the design space.

Specifically, **approximate designs** specify the proportion of observations allocated to each support point through the weights w_i and are represented as in (3.2). Such designs are primarily used because they simplify the optimization problem and the theory.

In practice, however, experiments require a finite number of observations, since the experimenter has to perform n_i experiments at each particular x_i .³ Therefore, the approximate weights must be converted into integer counts, producing what is known as an **exact design**.

²Jesús López-Fidalgo (2023). *Optimal Experimental Design: A Concise Introduction for Researchers*. Vol. 226. Lecture Notes in Statistics. Cham: Springer Nature Switzerland. ISBN: 978-3-031-35917-0. DOI: 10.1007/978-3-031-35918-7

³López-Fidalgo 2023.

Example 3.1

Support Points x_i	Weights ω_i
$x_1 = 0$	$\omega_1 = 0.3$
$x_2 = 20$	$\omega_2 = 0.4$
$x_3 = 100$	$\omega_3 = 0.3$

The table illustrates an example of an approximate design ξ specifying that:

- 30% of the sample is to be taken at $x = 0$
- 40% at $x = 20$
- 30% at $x = 100$

If the total sample size is $n = 10$, the ideal allocations are

$$10 \cdot 0.3 = 3, \quad 10 \cdot 0.4 = 4, \quad 10 \cdot 0.3 = 3.$$

Thus, the corresponding exact design becomes:

Support Points x_i	Counts n_i
$x_1 = 0$	$n_1 = 3$
$x_2 = 20$	$n_2 = 4$
$x_3 = 100$	$n_3 = 3$

Meaning that:

- $x = 0$ is repeated 3 times,
- $x = 20$ is repeated 4 times,
- $x = 100$ is repeated 3 times,

with $n_1 + n_2 + n_3 = n$.

□

For a given model with parameter vector $\boldsymbol{\theta}$, the central problem of optimal design experiments is to determine a design ξ^* that yields the most precise estimate of $\boldsymbol{\theta}$. To achieve that, ξ^* must be a design for which the information about the parameter is *maximized*.

3.1.2 Fisher Information

The natural question becomes:

"How does one quantify the informative quality of a design?"

A fundamental tool in design theory for this purpose is the **Fisher Information Matrix (FIM)**. Throughout this section, standard regularity conditions ensuring the validity of inference based on the likelihood function and Fisher information theory are assumed to be satisfied.

Suppose that the response variable Y follows a probability distribution with density $f(y; \boldsymbol{\theta})$, where $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^p$ is an unknown parameter vector. Given observations $\mathbf{y} = (y_1, \dots, y_n)$, the **likelihood function** is defined as

$$L(\boldsymbol{\theta}; \mathbf{y}) = \prod_{i=1}^n f(y_i; \boldsymbol{\theta}),$$

and the **log-likelihood function** is

$$\ell(\boldsymbol{\theta}; \mathbf{y}) = \log L(\boldsymbol{\theta}; \mathbf{y}).$$

The **score vector** is defined as the gradient of the log-likelihood:⁴

$$V(\boldsymbol{\theta}; \mathbf{y}) = \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}; \mathbf{y}) = \left(\frac{\partial}{\partial \theta_1} \ell(\boldsymbol{\theta}; \mathbf{y}), \dots, \frac{\partial}{\partial \theta_p} \ell(\boldsymbol{\theta}; \mathbf{y}) \right)^T \quad (3.3)$$

This function measures the relative rate at which the likelihood function $\mathcal{L}(\cdot; \mathbf{y})$ changes as a function of $\boldsymbol{\theta}$. I.e., it can be interpreted as a measure of how *sensitive* the log-likelihood is locally to changes in the parameter.⁵ Consequently, it quantifies the local direction of increase of the log-likelihood.

Thus, if the true parameter is $\boldsymbol{\theta}_0$ and $V(\boldsymbol{\theta}_0; \mathbf{y})$ is large, it is easier to distinguish likely values of $\boldsymbol{\theta}_0$ from neighboring unlikely values. In this sense, the larger $\mathbb{E}_{\boldsymbol{\theta}}[V(\boldsymbol{\theta}; \mathbf{Y})V(\boldsymbol{\theta}; \mathbf{Y})^T]$ is, the more informative is $\mathbf{Y}$ on the underlying $\boldsymbol{\theta}$. Moreover, the expected value of the score function can be shown to be zero, reflecting that the gradient of the log-likelihood function at the true parameter value, when averaged over all possible observations, is maximized and does not favor any specific direction.⁶

⁴Hannelore Liero and Silvelyn Zwanzig (2006). *Introduction to the Theory of Statistical Inference*. Springer

⁵Liero and Zwanzig 2006.

⁶Liero and Zwanzig 2006.

Hence, the **Fisher Information Matrix** (FIM) is defined as the following $p \times p$ covariance matrix⁷:

$$\mathcal{I}(\boldsymbol{\theta}) = \text{Cov}_{\boldsymbol{\theta}}[V(\boldsymbol{\theta}; \mathbf{Y})] = \mathbb{E}_{\boldsymbol{\theta}}[V(\boldsymbol{\theta}; \mathbf{Y})V(\boldsymbol{\theta}; \mathbf{Y})^T] \quad (3.4)$$

The Fisher information can also be expressed as a measure of the **expected curvature** of the (log-)likelihood function.

Let $\mathcal{J}(\boldsymbol{\theta}; \mathbf{Y})$ be the matrix of negative second derivatives of the log-likelihood function, also known as the **observed information** matrix, meaning that it has elements of the form:

$$\mathcal{J}(\boldsymbol{\theta}; \mathbf{Y})_{jr} = -\frac{\partial^2 \ell(\boldsymbol{\theta}; \mathbf{Y})}{\partial \theta_j \partial \theta_r} \quad j, r = 1, 2, \dots, p$$

Then, **equivalently**, the FIM can be represented as⁷:

$$\mathcal{I}(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\theta}}[\mathcal{J}(\boldsymbol{\theta}; \mathbf{Y})]. \quad (3.5)$$

The interpretation is equivalent to that given by the score: a sharply peaked likelihood surface corresponds to high information, while a flat likelihood indicates low information.

The primary purpose of the FIM in experimental design studies is to establish a limit of optimal precision for parameter estimation. Let $\hat{\boldsymbol{\theta}}$ and $\tilde{\boldsymbol{\theta}}$ be **unbiased** estimators for $\boldsymbol{\theta}$. Then, $\hat{\boldsymbol{\theta}}$ is said to have a smaller covariance matrix than $\tilde{\boldsymbol{\theta}}$ at $\boldsymbol{\theta} \in \Theta$ if $\text{Cov}_{\boldsymbol{\theta}}(\hat{\boldsymbol{\theta}}) - \text{Cov}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{\theta}})$ is **positive semidefinite**. Denote this by:

$$\text{Cov}_{\boldsymbol{\theta}}(\hat{\boldsymbol{\theta}}) \preceq \text{Cov}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{\theta}})$$

Then, the **Cramér–Rao inequality** states that the covariance matrix of any unbiased estimator $\hat{\boldsymbol{\theta}}$ satisfies⁸:

$$\mathcal{I}(\boldsymbol{\theta})^{-1} \preceq \text{Cov}_{\boldsymbol{\theta}}(\hat{\boldsymbol{\theta}}) \quad (3.6)$$

which means that the inverse of the Fisher information matrix provides a **lower bound** on the achievable variance of the estimator.

⁷Liero and Zwanzig 2006.

⁸Liero and Zwanzig 2006.

An unbiased estimator of particular interest is the **Maximum Likelihood Estimator (MLE)**, $\hat{\boldsymbol{\theta}}_{MLE}$. In fact, it can be shown that

$$\hat{\boldsymbol{\theta}}_{MLE} \approx \mathcal{N}(\boldsymbol{\theta}_0, \mathcal{I}(\boldsymbol{\theta}_0)^{-1}) \quad (3.7)$$

for large sample size n , meaning that it is *asymptotically normal and efficient*.

That being said, the FIM also has an important geometric interpretation in this context. Given that $\mathcal{I}(\boldsymbol{\theta}_0)^{-1}$ is the covariance matrix of the asymptotic distribution, the **confidence ellipsoid** for $\hat{\boldsymbol{\theta}}_{MLE}$ is approximated by:

$$(\hat{\boldsymbol{\theta}}_{MLE} - \boldsymbol{\theta})^T \mathcal{I}(\boldsymbol{\theta}_0) (\hat{\boldsymbol{\theta}}_{MLE} - \boldsymbol{\theta}) \leq c$$

with shape and orientation determined by the eigenvalues and eigenvectors of $\mathcal{I}(\boldsymbol{\theta}_0)^{-1}$.

Moreover, in multivariate statistical analysis, the determinant of the covariance matrix, often referred to as the **generalized variance**, provides a measure of the overall variability of a random vector. It can be shown that this determinant is proportional to the square of the volume generated by the deviation vectors, implying that the volume of the corresponding ellipsoidal region is proportional to the square root of the determinant.⁹

Consequently, in the context of asymptotic normality of the MLE, with covariance matrix approximated by the inverse of the FIM:

The volume of the confidence ellipsoid is **proportional** to $\det[\mathcal{I}(\boldsymbol{\theta}_0)^{-1}]^{1/2}$ and therefore inversely proportional to $\sqrt{\det[\mathcal{I}(\boldsymbol{\theta}_0)]}$: the larger the determinant of the FIM, the smaller the volume of the ellipsoid. It is exactly this relationship that motivates the D-optimality criterion.

3.1.3 D-Optimal Design

In optimal design theory, the Fisher Information Matrix is expressed as a function of the experimental design

$$\boldsymbol{\xi} = \{(x_i, w_i)\}_{i=1}^k,$$

⁹Richard A. Johnson and Dean W. Wichern (2014). *Applied Multivariate Statistical Analysis*. 6th ed. Harlow, Essex: Pearson Education Limited. ISBN: 978-1-292-02494-3

where $x_i \in \mathcal{X}$ are support points and $w_i \geq 0$ are weights s.t. $\sum_{i=1}^k w_i = 1$.

Assuming independent observations, the **Fisher Information Matrix** of a design ξ is given by¹⁰

$$\mathcal{I}(\boldsymbol{\theta}, \xi) = \sum_{i=1}^k w_i \cdot g(x_i, \boldsymbol{\theta}) g(x_i, \boldsymbol{\theta})^T = \sum_{i=1}^k w_i \cdot \mathcal{I}(\boldsymbol{\theta}; x_i). \quad (3.8)$$

where $g(x; \boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \eta(x; \boldsymbol{\theta})$.

Among many optimality criteria, the **D-optimality criterion** seeks to find the design that **minimizes the volume of the joint confidence ellipsoid** about the p parameters in $\boldsymbol{\theta}$.¹¹ Therefore, a **D-optimal design** ξ^* is a design with experimental settings (support points and weights) that **maximize the determinant of the FIM**, that is¹²:

$$\xi^* = \arg \max_{\xi} \det[\mathcal{I}(\xi, \boldsymbol{\theta})] \quad (3.9)$$

As discussed in the previous section, when considering asymptotically efficient estimators, (3.9) implies that a D-optimal design minimizes the volume of the confidence ellipsoid associated with the parameter estimates. Hence, D-optimality aims to reduce the range of plausible parameter values and make the joint uncertainty of, e.g., Maximum Likelihood estimates as small as possible.

In practice, it is common to **minimize**:

$$\Psi(\xi) = -\log[\det[\mathcal{I}(\xi, \boldsymbol{\theta})]] \quad (3.10)$$

instead of directly maximizing $\det[\mathcal{I}(\xi, \boldsymbol{\theta})]$. The approaches are equivalent because the logarithm function is strictly increasing.¹³ Therefore, throughout this thesis, $\Psi(\xi)$ will be used as the **objective function** to be minimized

¹⁰Jesús López-Fidalgo (2023). *Optimal Experimental Design: A Concise Introduction for Researchers*. Vol. 226. Lecture Notes in Statistics. Cham: Springer Nature Switzerland. ISBN: 978-3-031-35917-0. DOI: 10.1007/978-3-031-35918-7

¹¹Goos and Jones 2011.

¹²Goos and Jones 2011.

¹³Mohamed Tarek, Jose Storopoli, and Joel Owen (2024). *Introduction to Optimal Design*. IntroductiontoOptimalDesigntutorial. Tutorial document

to determine D-optimal designs.

However, a circular problem arises: since $\mathcal{I}(\boldsymbol{\theta}; \xi)$ is dependent on the parameter, a known value of $\boldsymbol{\theta}$ is required to be able to compute it; thus, a guess must be specified *a priori* in order to find an optimal design, but an optimal design is needed to estimate the unknown $\boldsymbol{\theta}$.

To resolve this issue, the Fisher Information Matrix is evaluated at a fixed nominal value $\boldsymbol{\theta} = \boldsymbol{\theta}_0$, typically obtained from prior knowledge or literature, and the goal becomes to find a design ξ^* that maximizes $\det(\mathcal{I}(\boldsymbol{\theta}_0; \xi))$. In other words, the design is optimal under the assumption that the true parameter lies in a neighborhood of $\boldsymbol{\theta}_0$; such a design is referred to as a **locally D-optimal design**.¹⁴ This is the approach that is adopted in this thesis.

3.1.4 The General Equivalence Theorem

Since D-optimal designs in nonlinear models are generally obtained through numerical optimization, it is important to verify the optimality of the proposed design. A fundamental result for this purpose is the General Equivalence Theorem (GET), originally established by Kiefer (1974).

Suppose that ξ^* is the design obtained from the optimization method of choice:

$$\xi^* = \{(x_1^*, w_1^*), (x_2^*, w_2^*), \dots, (x_k^*, w_k^*)\}.$$

The support points $x_i^* \in S_{\xi^*}$ have been picked from the design space $\mathcal{X}$ by the optimization method. Therefore, they correspond to the design points in $\mathcal{X}$ with positive weights, while any $x_i \in \mathcal{X}$ not in S_{ξ^*} has zero weight.

Let

$$d(x, \xi) = g^T(x, \boldsymbol{\theta}) \mathcal{I}^{-1}(\xi) g(x, \boldsymbol{\theta}) \quad (3.11)$$

be the **sensitivity function**, where $g(x, \boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \eta(x, \boldsymbol{\theta})$. The corresponding **directional derivative** is defined as

$$\Phi(x, \xi) = d(x, \xi) - p, \quad (3.12)$$

¹⁴Tarek, Storopoli, and Owen 2024.

where p is the number of parameters.

The **General Equivalence Theorem** can be stated as follows¹⁵:

A design ξ^* is D-optimal if and only if

$$\Phi(x, \xi^*) \leq 0 \quad \text{for all } x \in \mathcal{X}. \quad (3.13)$$

Moreover, equality holds at every support point of the optimal design, that is,

$$\Phi(x_i, \xi^*) = 0 \quad (3.14)$$

for all x_i such that $w_i > 0$.

The original proof is due to Kiefer (1974).

Interpretation. The idea is that moving some small measure from the support set S_{ξ^*} decreases $\Phi(x, \xi^*)$, in other words, $\Phi(x, \xi^*)$ indicates points at which an observation contributes most to approaching the optimal design.¹⁶

When plotting the directional derivative, the curve should stay below 0 or be flat at the support points when equal to 0, meaning that the design cannot be improved anywhere in the design space by adding weight at some point.

OBS. The theorem can equivalently be expressed as

$$d(x, \xi^*) \leq p \quad \text{for all } x \in \mathcal{X} \quad (3.15)$$

and

$$d(x_i^*, \xi^*) = p \quad \text{at every support point of } \xi^* \quad (3.16)$$

The approach that will be used in this study is to exploit *metaheuristic* optimization algorithms; these will be introduced in later sections. The GET will be crucial for verifying whether the designs obtained from these algorithms

¹⁵Atkinson, Donev, and Tobias 2007.

¹⁶Fedorov and Hackl 2025.

truly are optimal or nearly optimal.

Example 3.2

Figure 3.1 shows an equivalent illustration of the interpretation above for a quadratic model in one variable with $\mathcal{X} = [-1, 1]$,

$$\xi^* = \left\{ \left(-1, \frac{1}{3} \right), \left(0, \frac{1}{3} \right), \left(1, \frac{1}{3} \right) \right\}$$

and $d(x, \xi^*)$ satisfying (3.15) and (3.16)¹⁷.

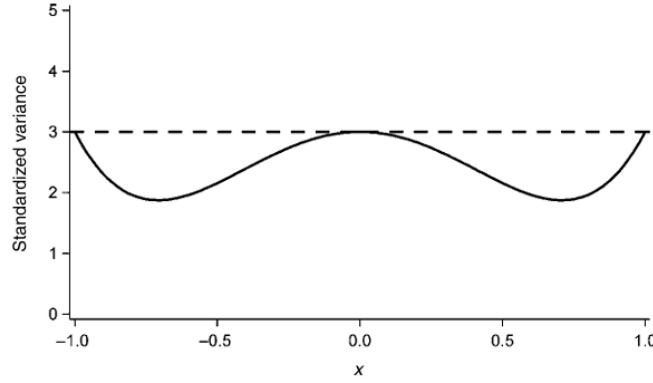


Figure 3.1: Plot of $d(x, \xi^*)$ for the D-optimal design. The maximum value of 3 occurs at the support points.

□

3.2 Models

This section introduces the nonlinear regression models of interest for the study, with their respective mean response function $\eta(x, \boldsymbol{\theta})$ and parameter vector $\boldsymbol{\theta}$ to be estimated. These will be embedded in the established nonlinear regression framework and set the blueprint for the upcoming results.

¹⁷Atkinson, Donev, and Tobias 2007.

3.2.1 Michaelis-Menten Model

One of the most fundamental nonlinear models is the Michaelis-Menten model, originally introduced by Michaelis and Menten (1913) as an enzyme kinetics model.¹⁸ It describes the relationship between a continuous response and a non-negative explanatory variable, such as substrate concentration or drug dose.

Its general form in pharmacological modeling is the **E_{max} model** given by:

$$y_i = E_0 + E_{\max} \frac{x_i}{ED_{50} + x_i} + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

where E_0 represents the baseline response when drug exposure is zero, $E_{\max}$ denotes the maximum change from baseline as the drug exposure becomes arbitrarily large, and ED_{50} is exposure level at which the change from baseline is 50% of the maximum possible change.¹⁹ In many applications, it is natural to assume $E_0 = 0$, which reduces the model to

$$y_i = E_{\max} \frac{x_i}{ED_{50} + x_i} + \epsilon_i.$$

This model is known as the uniresponse Emax model but is often referred to as the **Michaelis-Menten model**.²⁰

This thesis, however, adopts the following equivalent parameterization

$$\eta(x, \boldsymbol{\theta}) = \frac{Vx}{K + x}, \quad (3.17)$$

where $V = E_{\max}$ and $K = ED_{50}$. The parameter $V > 0$ represents the maximal attainable response, while $K > 0$ denotes the input value at which half of this maximum is achieved. Assuming known variance of the errors ϵ_i , the parameter vector for this model is therefore $\boldsymbol{\theta} = (V, K)$.

The structure of the function shows a clear saturation characteristic behaviour: For small values of x , the response increases approximately linearly,

¹⁸Kim A. Keating and James F. Quinn (1998). “Estimating species richness: The Michaelis-Menten model revisited”. In: *Oikos* 81.2, pp. 411–416

¹⁹Danielle Navarro (Nov. 11, 2024). *Bayesian Estimation for Emax Regression*. URL: https://blog.djnavarro.net/posts/2024-11-11_emax-parameters/

²⁰Magnusdottir 2016.

while as $x \rightarrow \infty$, the response approaches the limiting value V . Graphically, V “stretches” the curve vertically, more precicely, increasing V shifts the asymptotic value for the response, while K is a shift parameter that moves the whole curve rightwards if increased (more examples in Chapter 4).²¹

Example 3.3

Figure 3.2 shows the curve generated from the mean response function $\eta(x, \theta)$ of the Michaelis-Menten model with parameter values $V = 15$ and $K = 6$.

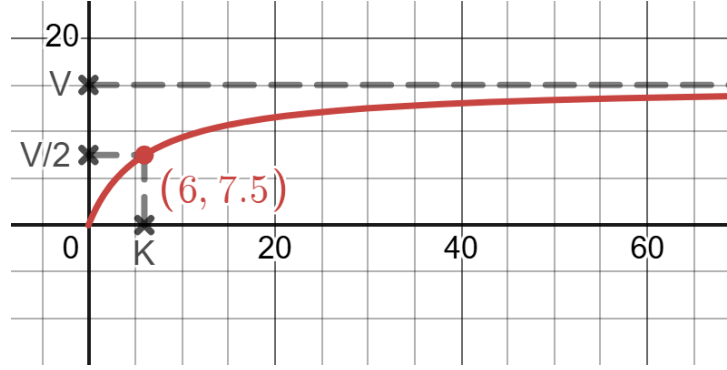


Figure 3.2: Plot of $\eta(x, \theta)$ in Example 3.3.

□

Later, in the practical part of the thesis, the design space $\mathcal{X} = [0, x_{max}]$ to be considered for this study will need to be defined by providing a reasonable value for the upper bound x_{max} . The choice of x_{max} for the 2-parameter Michaelis-Menten model is defined here as:

$$x_{max} = 19 \cdot K \quad (3.18)$$

to ensure that a wide range of design points is covered. This is justified by equation (3.17) itself:

Divide both sides by V

$$\frac{\eta(x; \theta)}{V} = \frac{x}{K + x}$$

²¹Navarro 2024.

When $x = 19K$

$$\frac{\eta(19K; \boldsymbol{\theta})}{V} = \frac{19K}{K + 19K} = \frac{19}{20} = 0.95$$

which means that the function value corresponds to exactly 95% of the maximum response. Hence, setting $x_{max} = 19K$ ensures exploration of a wide space including the region where the response approaches its maximum.

3.2.2 Emax Model

The Emax model is a widely used nonlinear regression model in pharmacodynamics and dose-response analysis. It is commonly used to model the effect of a drug at a given concentration, and it generalizes the Michaelis-Menten model by introducing an additional parameter that controls the steepness of the response curve.

In its general form, the **Emax model** is given by

$$\eta(x, \boldsymbol{\theta}) = E_0 + E_{\max} \frac{x^h}{ED_{50}^h + x^h},$$

where $x \geq 0$ denotes the concentration, and $\boldsymbol{\theta} = (E_0, E_{\max}, ED_{50}, h)^T$ is the parameter vector. The additional parameter $h > 0$ is the **Hill coefficient** controlling the steepness of the response curve.²²

The Emax model describes a relationship in which the response increases with x but at a decreasing rate and approaches an asymptotic limit. In particular, as $x \rightarrow \infty$, the response approaches $E_0 + E_{\max}$.²³

An important feature of the Emax model is the Hill coefficient h , which provides additional flexibility compared to the Michaelis–Menten model. It is not uncommon to fix $h = 1$, then, the model reduces to the Michaelis–Menten form. For $h > 1$, the response curve becomes steeper around the point $x = EC_{50}$, while for $0 < h < 1$, the curve becomes more gradual.²⁴

²²Navarro 2024.

²³Navarro 2024.

²⁴Navarro 2024.

Let $E_0 = 0$, and denote $V = E_{\max}$ and $K = ED_{50}$. The following parameterization will be used for the Emax model in the actual framework:

$$\eta(x, \boldsymbol{\theta}) = \frac{Vx^h}{K^h + x^h} \quad (3.19)$$

$V > 0$, $K > 0$ and $h > 0$. Thus, in this case, the parameter vector to be considered is $\boldsymbol{\theta} = (V, K, h)$.

Example 3.4 Figure 3.3 shows the curve generated from the mean response function $\eta(x, \boldsymbol{\theta})$ of the Emax model with parameter values $V = 15$, $K = 6$ and $h = 4$.

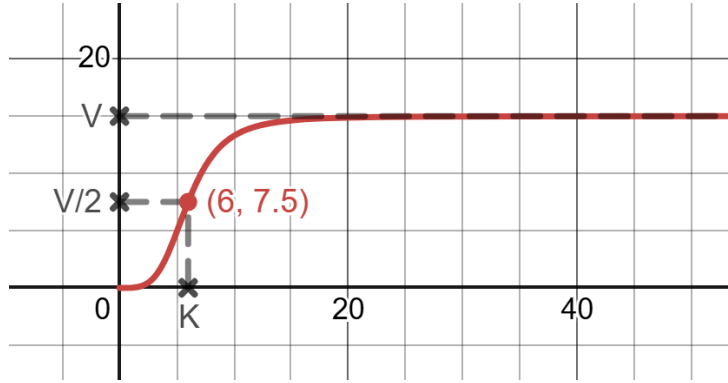


Figure 3.3: Plot of $\eta(x, \boldsymbol{\theta})$ in Example 3.4.

□

Here, the upper bound $x_{\max}$ of the design space for the 3-parameter Emax model is defined for later use as:

$$x_{\max} = (19K)^{\frac{1}{h}} \quad (3.20)$$

The idea is analogous to the above, for Michaelis-Menten. At $x = cK$, dividing both sides of (3.19) by V yields

$$\frac{\eta(cK; \boldsymbol{\theta})}{V} = \frac{c^h}{1 + c^h}$$

To reach a specified percentage of saturation $100 \cdot q\%$, solve:

$$q = \frac{c^h}{1 + c^h} \Rightarrow c = \left(\frac{q}{1 + q} \right)^{\frac{1}{h}}$$

Setting $q = 0.95$

$$x_{max} = (19K)^{\frac{1}{h}}$$

Note that (3.18) is simply the case when $h = 1$.

3.3 Metaheuristics

The following optimization algorithms will be used to obtain near optimal designs according to the local D-optimality criterion. Both of them belong to the class of nature-inspired metaheuristics and are characterized by their ability to handle complex optimization problems without requiring strong assumptions such as differentiability of the objective function. They set the stage for the main purpose of this study, which is a comparative analysis of their performance.

3.3.1 Particle Swarm Optimization

Particle Swarm Optimization (PSO) is population-based metaheuristic originally introduced by Kennedy and Eberhart in the mid-1990s and inspired by the swarming and flocking behaviors in animals, such as bird flocks.²⁵ It has since become a widely used method for solving complex optimization where the objective function is often nonlinear and may contain multiple local optima.

The Idea. In PSO, a set of candidate solutions, called particles, move through the search space in order to find an optimal or near-optimal solution. Each particle represents a point in the search space and is assigned a velocity that determines how it moves. The movement of particles is influenced by both their personal best found position and the global best position of the entire swarm.

²⁵Sean Luke (2009). *Essentials of Metaheuristics*. Available online at <http://cs.gmu.edu/~sean/book/metaheuristics/>. Lulu

Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ denote the entire swarm, where n is the number of particles and d is the length of each particle, and let Ψ be the objective function. Further, let $\mathbf{X}_i^{(t)} \in \mathbb{R}^d$ denote the position of particle i at iteration t , and let $\mathbf{V}_i^{(t)}$ denote its velocity. Each particle represents a candidate design. A position $\mathbf{p}_i$ for the fixed particle i is a personal best if

$$\Psi(\mathbf{p}_i) = \min_t \Psi(\mathbf{X}_i^{(t)})$$

while $\mathbf{p}_g$ is the global best at iteration t if

$$\Psi(\mathbf{p}_g) = \min \Psi(\mathbf{X}^{(t)})$$

Each particle keeps track of:

- its personal best position $\mathbf{p}_i$, and
- the global best position $\mathbf{p}_g$ found by the swarm.

Then, the velocity and position updates are given by

$$\mathbf{V}_i^{(t+1)} = w\mathbf{V}_i^{(t)} + c_1\mathbf{r}_1 \circ (\mathbf{p}_i - \mathbf{X}_i^{(t)}) + c_2\mathbf{r}_2 \circ (\mathbf{p}_g - \mathbf{X}_i^{(t)}), \quad (3.21)$$

and

$$\mathbf{X}_i^{(t+1)} = \mathbf{X}_i^{(t)} + \mathbf{V}_i^{(t+1)}, \quad (3.22)$$

where w is the **inertia weight**, c_1 and c_2 are **acceleration coefficients**, and $\mathbf{r}_1, \mathbf{r}_2 \sim \text{Uniform}(0, 1)$ are vectors consisting of random numbers. The product in the last two terms of (3.21), “ $\circ$ ”, is the Hadamard product, which multiplies two vectors element-wise.²⁶

The inertia term controls the influence of the former velocity to $\mathbf{V}_i^{(t+1)}$, while the constants c_1 and c_2 adjust the strides of the particles’ movement toward each of its personal best position and the global best position.²⁷

²⁶Ping-Yang Chen, Ray-Bing Chen, and Weng Kee Wong (2022). “Particle swarm optimization for searching efficient experimental designs: A review”. In: *WIREs Computational Statistics*, e1578. DOI: 10.1002/wics.1578

²⁷P.-Y. Chen, R.-B. Chen, and Wong 2022.

Algorithm. The PSO algorithm can be summarized as follows:

1. Initialize a swarm of particles:
 - a) Initiate random positions $\mathbf{X}_i$ and velocities $\mathbf{V}_i$ for $i = 1, \dots, n$.
2. Evaluate the objective function for each particle and store $\mathbf{p}_i$ and $\mathbf{p}_g$:
 - a) Calculate the objective function values $\Psi(\mathbf{X}_i)$ for $i = 1, \dots, n$.
 - b) Set initial personal best positions $\mathbf{p}_i$ and global best $\mathbf{p}_g$.
3. Repeat until a stopping criterion is met (e.g., maximum number of iterations):
 - a) Update velocities using the velocity update equation (3.21).
 - b) Update particle positions via equation (3.22).
 - c) Evaluate the objective function at the updated particles $\Psi(\mathbf{X}_i)$.
 - d) Update personal bests $\mathbf{p}_i$ and global best $\mathbf{p}_g$.
4. Output $\mathbf{p}_g = \arg \min \Psi(\mathbf{X})$ with $gbest = \Psi(\mathbf{p}_g)$.²⁸

3.3.2 Grey Wolf Optimizer

Grey Wolf Optimizer (GWO) is a modern population-based metaheuristic algorithm inspired by the social hierarchy and hunting behavior of grey wolves in nature. The algorithm was originally proposed by S. Mirjalili, S. M. Mirjalili, and Lewis (2014) and belongs to the class of swarm intelligence methods. Similar to other metaheuristics, GWO is characterized by its simplicity, flexibility, and derivative-free nature, making it suitable for solving complex optimization problems.²⁹

²⁸P.-Y. Chen, R.-B. Chen, and Wong 2022.

²⁹Seyedali Mirjalili, Seyed Mohammad Mirjalili, and Andrew Lewis (2014). “Grey Wolf Optimizer”. In: *Advances in Engineering Software* 69, pp. 46–61. DOI: 10.1016/j.advengsoft.2013.12.007

The Idea. In GWO, candidate solutions are represented as grey wolves, which are organized according to a social hierarchy. The “ best solution found so far ” is denoted as the alpha (α), and the second and third best solutions are called beta (β) and delta (δ), respectively.³⁰ Specifically,

- the α wolf $\mathbf{X}_\alpha^{(t)}$ is defined as

$$\mathbf{X}_\alpha^{(t)} = \arg \min_i \Psi(\mathbf{X}_i^{(t)})$$

$$i = 1, \dots, n_{wolves};$$

- the β wolf $\mathbf{X}_\beta^{(t)}$ is the solution with the second smallest objective value;
- the δ wolf $\mathbf{X}_\delta^{(t)}$ is the solution with the third smallest objective value;

where t indicates the current iteration, Ψ denotes the objective function, $\mathbf{X}_i$ is the position vector of a grey wolf, and n_{wolves} is the number of wolves.

The remaining candidate solutions are labeled omega (ω). The hunting (optimization process) is primarily guided by the positions of the α , β , and δ wolves, which together influence the movement of the rest of the pack.³¹

The algorithm mimics the main phases of grey wolf hunting: encircling the prey, hunting, and attacking the prey. The encircling process is modeled mathematically by the following position update equation:

$$\begin{aligned} \mathbf{D} &= |\mathbf{C}\mathbf{X}_p^{(t)} - \mathbf{X}_i^{(t)}| \\ \mathbf{X}_i^{(t+1)} &= \mathbf{X}_p^{(t)} - \mathbf{A}\mathbf{D} \end{aligned} \tag{3.23}$$

where $\mathbf{A}$ and $\mathbf{D}$ are coefficient vectors and $\mathbf{X}_p$ is the position vector of the prey. $\mathbf{A}$ and $\mathbf{D}$ are calculated as follows:

$$\begin{aligned} \mathbf{A} &= 2a\mathbf{r}_1 - \mathbf{a} \\ \mathbf{D} &= 2 \cdot \mathbf{r}_2 \end{aligned}$$

where the components of $\mathbf{a}$ are linearly decreased from 2 to 0 over the course of iterations and $\mathbf{r}_1, \mathbf{r}_2 \sim \text{Uniform}(0, 1)$ are random vectors.³²

³⁰S. Mirjalili, S. M. Mirjalili, and Lewis 2014.

³¹S. Mirjalili, S. M. Mirjalili, and Lewis 2014.

³²S. Mirjalili, S. M. Mirjalili, and Lewis 2014.

Assuming that the positions $\mathbf{X}_\alpha$, $\mathbf{X}_\beta$ and $\mathbf{X}_\delta$ of the α , β and δ wolf respectively are the best estimated locations of the prey, three alternative position updates are computed for wolf i as follows:

$$\begin{aligned} D_\alpha &= |C_1 \mathbf{X}_\alpha - \mathbf{X}_i|, & D_\beta &= |C_2 \mathbf{X}_\beta - \mathbf{X}_i|, & D_\delta &= |C_3 \mathbf{X}_\delta - \mathbf{X}_i| \\ \mathbf{X}_1 &= \mathbf{X}_\alpha - A_1 D_\alpha, & \mathbf{X}_2 &= \mathbf{X}_\beta - A_2 D_\beta, & \mathbf{X}_3 &= \mathbf{X}_\delta - A_3 D_\delta \end{aligned}$$

The updated position of wolf i is the average of the three best candidate updates³³

$$\mathbf{X}_i^{(t+1)} = \frac{\mathbf{X}_1^{(t+1)} + \mathbf{X}_2^{(t+1)} + \mathbf{X}_3^{(t+1)}}{3} \quad (3.24)$$

Algorithm. The GWO algorithm can be summarized as follows:

1. Initialize the grey wolf population:
 - a) Generate N random candidate solutions $\mathbf{X}_i$, $i = 1, \dots, N$.
2. Calculate fitness of all wolves and determine α , β and δ :
 - a) Calculate the objective function values $\Psi(\mathbf{X}_i)$ for $i = 1, \dots, N$.
 - b) Identify the three best solutions:
 - $\mathbf{X}_\alpha$ = best position
 - $\mathbf{X}_\beta$ = second best position
 - $\mathbf{X}_\delta$ = third best position
3. Repeat until a stopping criterion is met (e.g., maximum number of iterations):
 - a) Update $\mathbf{a}$, which decreases linearly over iterations.
 - b) For each wolf $\mathbf{X}_i$:
 - Generate random vectors $\mathbf{r}_1, \mathbf{r}_2 \sim \text{Uniform}(0, 1)$
 - Compute coefficient vectors $\mathbf{A}$ and $\mathbf{C}$
 - Calculate the distances to the three best wolves α , β and δ and the three candidate positions based on $\mathbf{X}_\alpha$, $\mathbf{X}_\beta$ and $\mathbf{X}_\delta$

³³S. Mirjalili, S. M. Mirjalili, and Lewis 2014.

- Update the position of $\mathbf{X}_i$ as the average of these three positions.
- c) Evaluate the objective function for the updated wolves $\Psi(\mathbf{X}_i)$.
- d) Update the hierarchy:
 - Update $\mathbf{X}_\alpha$, $\mathbf{X}_\beta$ and $\mathbf{X}_\delta$ based on the new objective values.
- 4. Return the best solution found, $\mathbf{X}_\alpha$, with its objective value $\Psi(\mathbf{X}_\alpha)$.

Chapter 4

Approach

4.1 Use of Artificial Intelligence

Artificial Intelligence (AI) tools have been used as supplementary aids during the work on this thesis. However, the author remains fully responsible for the mathematical formulations, choices of methods and approach, implementation of the codes, analysis, interpretation of results, and final written content.

AI assistance was primarily used in the following ways:

1. **Planning and implementation guidance.** AI was used to generate structured step-by-step plans and implementation guides for the programming tasks. These guides were used to provide a clear structure of the workflow by specifying the possible procedures, creating pseudo-codes, etc. The source code itself was designed, implemented, tested, and modified by the author.
2. **Programming assistance.** GitHub Copilot was used within Visual Studio Code to provide code-completion suggestions and programming assistance. Suggested lines of code were adapted when necessary, and validated by the author before being included in the final implementation.
3. **Language and writing support.** AI tools were occasionally used to improve grammar, clarity, and academic style in some parts of the written text. In some cases, AI suggestions were used to help formulate

ideas that had already been developed by the author. All text included in the final thesis was reviewed, edited, and approved by the author.

AI tools were not used to generate the mathematical results, perform the statistical analyses, or replace the author’s own understanding of the methods and results presented in this thesis.

4.2 Setup and Workflow

Disclaimer: All computational work was implemented in Python.

Initially, the goal is to be able to find a D-optimal design ξ for both the Michaelis-Menten model and the general Emax model using the metaheuristics defined in the previous chapter. For this purpose, designs will be represented by arrays of the form

$$\mathbf{x_i} = [\mathbf{x_1}, \dots, \mathbf{x_k}, \mathbf{w_1}, \dots, \mathbf{w_k}], \quad \text{shape } (2k,)$$

containing support points and weights. Such a design array will correspond to a particle in the PSO procedure, and the position of a wolf in the GWO procedure.

OBS. The solution returned by each of the optimization algorithms is thus of the form of an approximate design.

Results from approximate design theory imply that optimal designs may be represented by

$$k \leq \frac{p(p+1)}{2} \quad \text{support points.}$$

Therefore, throughout the implementation, the **number of candidate support points** is chosen as

$$\boxed{k = \frac{p(p+1)}{2}} \tag{4.1}$$

giving $k = 3$ for the Michaelis–Menten model and $k = 6$ for the Emax model.

x_{max} is a user-provided value for the upper bound of the design space. Recall that the Michaelis-Menten model has parameters V and K , while the more general Emax model has an additional parameter h . In order to cover a wide range of informative design points, including the region at which $\eta(x; \boldsymbol{\theta})$ approaches its maximum, x_{max} is defined as

$$x_{max} = \begin{cases} 19K, & \text{if } p = 2 \quad (\text{Michaelis-Menten}) \\ (19K)^{\frac{1}{h}}, & \text{if } p = 3 \quad (\text{Emax}) \end{cases} \quad (4.2)$$

given by the formulas (3.18) and (3.20) justified in section 3.2, where p is the number of parameters.

Throughout the work of this thesis, the following values of $\boldsymbol{\theta}$ will be considered as different cases for which designs and other results from PSO and GWO will be produced. Comparisons between the two metaheuristics will be based on them.

For the 2-parameter Michaelis-Menten (MM) model, $\boldsymbol{\theta} = (V, K)$:

- **Case 1:** $V = 3, K = 0.5$
- **Case 2:** $V = 7, K = 2$
- **Case 3:** $V = 5, K = 8$
- **Case 4:** $V = 5, K = 4$

Figure 4.1 illustrates these.

For the 3-parameter Emax model, $\boldsymbol{\theta} = (V, K, h)$:

- **Case 1:** $V = 1, K = 1, h = 0.5$
- **Case 2:** $V = 2.5, K = 2, h = 6$
- **Case 3:** $V = 2, K = 6, h = 2$
- **Case 4:** $V = 3, K = 0.2, h = 2$

Figure 4.2 illustrates these.

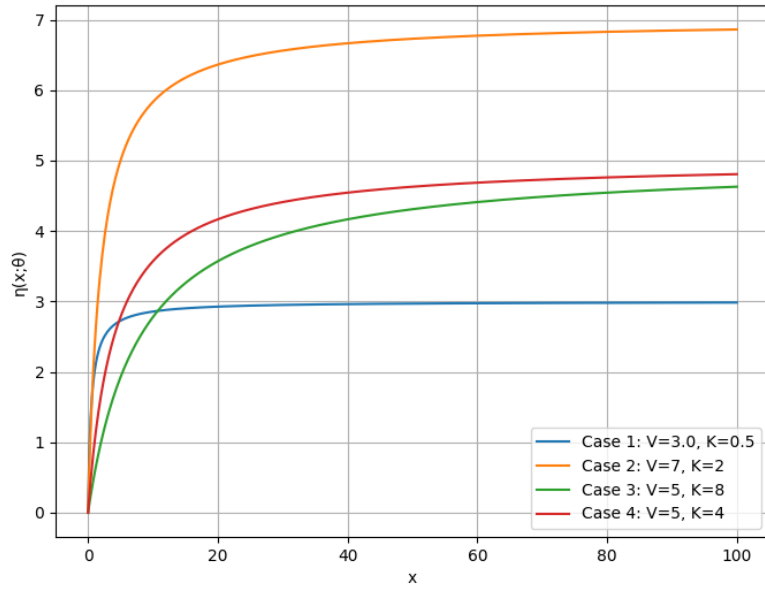


Figure 4.1: Michaelis-Menten model for different cases

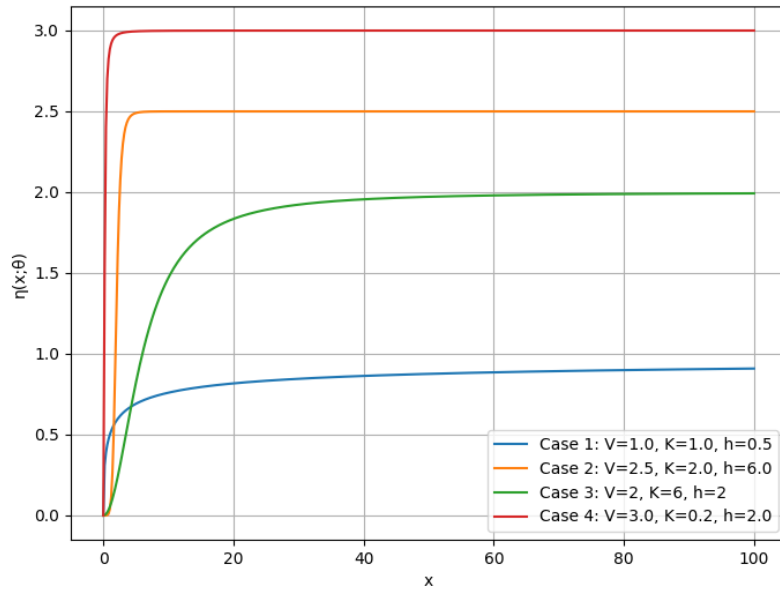


Figure 4.2: Emax model for different cases

After implementing the algorithms and obtaining D-optimal designs for all cases, the optimality of these solutions is verified through an application of the General Equivalence Theorem (GET). In more detail, the objective is to verify if the designs obtained from PSO and GWO *nearly* satisfy the criterion of D-optimality. Given a proposed design ξ^* , the GET implies that $\Phi(x_i^*, \xi^*)$ should be close to 0 if the support points x_i^* are near-optimal. Therefore, the **GET check** is considered a **SUCCESS** if ξ^* satisfies:

$$\boxed{\max \Phi(x, \xi^*) \leq \text{tol} \quad \forall x \in \mathcal{X} \quad \text{and} \quad |\Phi(x_i^*, \xi^*)| \leq \text{tol} \quad \forall x_i^*} \quad (4.3)$$

for some specified tolerance “tol”, $i = 1, 2, \dots, k$. Additionally, this section also takes care of the cleaning and compressing of the raw solutions obtained from the algorithms, including operations such as: dropping near-zero weights, merging duplicate support points and renormalizing weights.

Next, diagnostics are computed for the performative comparison of PSO and GWO by running both of them a large number of times, N . For efficiency, a multiprocessing approach with parallelization is used. At each run, the following values are stored for each of the two algorithms:

1. t : runtime in seconds.
2. n_{stop} : number of iterations needed to meet stopping criteria.
3. $\max_i |\Phi(x_i, \xi^*)|$: maximum of the absolute value of the directional derivative evaluated at the support points (should be close to 0 if support points are optimal).

After N runs, both PSO and GWO will yield:

$$\begin{aligned} &[t_1, t_2, \dots, t_N] \\ &[n_{stop_1}, n_{stop_2}, \dots, n_{stop_N}] \\ &[\max|\Phi|_1, \max|\Phi|_2, \dots, \max|\Phi|_N] \end{aligned}$$

for which the following diagnostics are computed to compare the performance of the two algorithms:

$$\max, Q_3, \text{median}, Q_1, \min, \text{mean}$$

Moreover, success rates are calculated by considering the number of times that:

(a) a **GET check** is **successful**, meaning that (4.3) is satisfied,

and the number of times that:

(b) a check at **support points only** is **successful**, meaning that the following is satisfied:

$$|\Phi(x_i^*, \xi^*)| \leq \text{tol} \quad \text{for all support points } x_i^*$$

which is equivalent to:

$$\boxed{\max_i |\Phi(x_i, \xi^*)| \leq \text{tol}} \quad (4.4)$$

These are counted for PSO and for GWO throughout the N runs to calculate their:

$$\text{Success Rate} = \frac{\# \text{ successful checks}}{N}$$

with respect to both (a) and (b).

The full process for computing these diagnostics is performed once for each of the parameter cases listed above.

The last part of the work consists in illustrating the efficiency of a D-optimal design. Given a **D-optimal design**, the goal is to compare it to the following arbitrarily chosen designs, with equidistant support points and equal weights, in terms of how well they estimate the parameter.

1. Non-Optimal Design - Scenario 1

$$\text{Support points: } \mathbf{x} = \left[0, \frac{x_{max}}{2}, x_{max} \right]$$

$$\text{Weights: } \mathbf{w} = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right]$$

2. Non-Optimal Design - Scenario 2

$$\text{Support points: } \mathbf{x} = \left[0, \frac{x_{max}}{3}, \frac{2 \cdot x_{max}}{3}, x_{max} \right]$$

$$\text{Weights: } \mathbf{w} = \left[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4} \right]$$

Based on the results of the previous section, the optimization method with the best performance will be chosen to compute the D-optimal design. The procedure is based on a Monte Carlo simulation approach, roughly explained by the following setup:

1. Generate data using optimal design ξ^* :

$$\begin{aligned} y_1^* &= \eta(x_1; \boldsymbol{\theta}) + \epsilon_1, & \epsilon_1 &\sim \mathcal{N}(0, \sigma^2) \\ &\vdots \\ y_n^* &= \eta(x_n; \boldsymbol{\theta}) + \epsilon_n, & \epsilon_n &\sim \mathcal{N}(0, \sigma^2) \end{aligned} \tag{4.5}$$

with e.g. $\sigma^2 = 0.5$, n is the sample size.

2. Similarly, generate data using non-optimal designs

$$y_j = \eta(x_j; \boldsymbol{\theta}) + \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, \sigma^2) \tag{4.6}$$

$j = 1, 2, \dots, n$.

3. Use Maximum Likelihood Estimator (MLE) to estimate $\boldsymbol{\theta}$ with each dataset.
4. Repeat a large number of times n_{sims} .

After, e.g. $n_{sims} = 1000$ runs, each design (D-optimal, non-optimal 1, non-optimal 2) will produce a set of estimates of the following form:

$$\left[\hat{\boldsymbol{\theta}}_1, \hat{\boldsymbol{\theta}}_1, \dots, \hat{\boldsymbol{\theta}}_{1000}, \right]$$

where $\hat{\boldsymbol{\theta}}_i$ is the MLE estimated in the i th run from the dataset of that run, $i = 1, 2, \dots, 1000$. A single parameter case is picked from each model framework for this experiment (case 1 for the MM model and case 4 for the Emax model), which is performed for different sample sizes $n \in [48, 72, 96]$.

Since the optimization method provides a representation of an approximate design $\xi^* = \{(x_i^*, w_i^*)\}_{i=1}^k$, an operation needed in step one is to convert this solution into an exact design. If n is the size of the sample to be collected, the ideal number of observations allocated at support point x_i^* is nw_i^* . Since these quantities are generally not integers, the values must be rounded in some manner while ensuring that the total number of observations remains equal to n .

In the implementation used in this thesis, the approximate counts nw_i^* are rounded down to the nearest integer $\lfloor nw_i^* \rfloor$, which is the exact number of

observations allocated at the support point x_i^* ; if the rounded counts do not sum to n , the remaining observations are allocated to the support points with the largest remainders. $nw_i^* - \lfloor nw_i^* \rfloor$. This procedure yields an exact design whose realizations are as close as possible to the specification of the original approximate design.

After converting weights into counts,

$$w_1^*, w_2^*, \dots, w_k^* \mapsto n_1, n_2, \dots, n_k \quad \text{s.t.} \quad \sum_{i=1}^k n_i = n$$

the sample $\mathbf{x} = [x_1, x_2, \dots, x_n]$ of size n is created accordingly and stored as an array. Then, for specified $\boldsymbol{\theta}$ and σ^2 , responses can be generated from the D-optimal design as in (4.5) by evaluating the model at the values of $\mathbf{x}$ and simulating the normally distributed random errors. A similar evaluation is done for the non-optimal designs.

Once responses have been generated under:

$$Y_j = \eta(x_j; \boldsymbol{\theta}) + \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, \sigma^2)$$

with independent errors ϵ_j and known variance σ^2 , the **Maximum Likelihood Estimator (MLE)** $\hat{\boldsymbol{\theta}}$ can be calculated from the simulated data. However, in this case, under the specified Gaussian noise assumption, estimation of $\boldsymbol{\theta}$ by maximum likelihood is equivalent to **Non-Linear Least Squares (NLS)** estimation, since maximizing the likelihood becomes equivalent to minimizing the sum of squared residuals. For this reason, $\boldsymbol{\theta}$ is estimated by **minimizing the Residual Sum of Squares (RSS)**

$$\text{RSS}(\boldsymbol{\theta}; \mathbf{y}) = \sum_{j=1}^n (y_j - \eta(x_j; \boldsymbol{\theta}))^2$$

Nevertheless, the model parameters must satisfy positivity constraints in order to be valid:

- For the MM model: $V > 0, K > 0$.
- For the Emax model: $V > 0, K > 0, h > 0$.

So, instead of optimizing over $\boldsymbol{\theta} \in \Theta$ directly, the idea is to optimize over a transformed parameter space where the constraints are automatically satisfied. The approach adopted in this thesis is to **optimize over log-transformed parameters**. New parameters $\phi_1, \dots, \phi_p$ are introduced, such that:

- For the MM model: $\phi_1 = \log(V)$, $\phi_2 = \log(K)$
- For the Emax model: $\phi_1 = \log(V)$, $\phi_2 = \log(K)$, $\phi_3 = \log(h)$

Then, the original parameters are recovered as:

$$V = \exp(\phi_1), K = \exp(\phi_2), h = \exp(\phi_3)$$

Since

$$\exp(\phi) > 0 \quad \text{for all } \phi$$

V , K and h are always ensured to be positive. Hence,

$$\min_{\boldsymbol{\theta} > 0} RSS(\boldsymbol{\theta}) \Leftrightarrow \min_{\boldsymbol{\phi}} RSS(\exp(\boldsymbol{\phi})) \quad (4.7)$$

where $\phi = \log(\theta)$. The function “`minimize()`” from `scipy.optimize` is used to perform the minimization. Briefly explained, it:

1. Starts at an initial guess $\boldsymbol{\phi}_0 = \log(\boldsymbol{\theta}_0)$.
2. Tries many candidate values of $\boldsymbol{\phi}$ by evaluating $RSS(\exp(\boldsymbol{\phi}))$ for each one of them.
3. Iteratively moves towards the $\boldsymbol{\phi}$ that minimizes the RSS.
4. Stops when no more meaningful improvement is found.

Since the original parameter $\boldsymbol{\theta} = [\theta_1, \theta_2, \dots, \theta_p]$ is known, a trivial initial guess $\boldsymbol{\theta}_0 \in \Theta$ would be $\boldsymbol{\theta}_0 = \boldsymbol{\theta}$. However, this may not reflect a realistic estimation. Instead, $\boldsymbol{\theta}_0 \in \Theta$ is defined as:

$$\boldsymbol{\theta}_0 = [\theta_1 \cdot \exp(Z_1), \dots, \theta_p \cdot \exp(Z_p)], Z_i \sim \mathcal{N}(0, 0.15^2) \quad (4.8)$$

$$i = 1, 2, \dots, p \quad \text{where } p \text{ is the number of parameters.}$$

That is, the true parameter is perturbed to yield a reasonable starting guess that is not too close to the actual $\boldsymbol{\theta}$ and to avoid making the optimization artificially easy. (4.8) is later transformed before being given as input to the

minimizer.

Thus, one simulation consists of generating data and minimizing the RSS to find the MLE $\hat{\theta}$, using each design (D-optimal, non-optimal 1, non-optimal 2); repeating this, say 1000 times, will lead to 1000 θ s for each design scenario ($\hat{\theta} = [\hat{V}, \hat{K}]$ for MM, $\hat{\theta} = [\hat{V}, \hat{K}, \hat{h}]$ for Emax). The last step is to plot these, using e.g. boxplots or histograms, to visualize and compare their distributions.

Since satisfying D-optimality corresponds to minimizing uncertainty of the estimator, the D-optimal design is expected to yield estimates with low variance (narrow spread) and more precise results, while the non-optimal designs are expected to yield estimates with larger variance (wide spread) and higher uncertainty.

The entire **workflow** can be summarized as follows:

- a) Implementation of PSO and GWO, and computation of D-optimal designs through them.
- b) Verification of optimality of the designs.
- c) Comparative analysis results for PSO and GWO.
- d) Response simulation and estimation of the parameters.

The full implementation has been split into five different modules, each tackling one of the steps listed above. These files have the following names and mapping to the workflow:

Loc_D_Opt.py $\rightarrow$ equivalence.py $\rightarrow$ OptimalityCheck.py $\rightarrow$
PSOvsGWO.py $\rightarrow$ ResponseSimulation.py

Each one of them can be found in the public GitHub repository attached to this document in the Appendix.

4.3 Implementation of PSO and GWO for D-Optimal Designs

4.3.1 Implementation Procedure

The first step is implementing the optimization algorithms. The Python file `Loc_D_Opt.py` has been created for this purpose. This module contains, in particular, the following functions.

Mean Response Function $\eta(x; \theta)$. The mean function of the response for both the Michaelis-Menten model and the general Emax model have been defined as `eta_mm(x, theta)` and `eta_emax(x, theta)` respectively, where `x` is an array of shape $(k,)$, and `theta` is an array of shape $(p,)$. The output is an array of the corresponding function values.

Fisher Information Matrix $\mathcal{I}(\theta; \xi)$. The function `FIM(xi, theta)` computes the Fisher information matrix of shape (p,p) using the gradient of the mean function that agrees with `p=len(theta)`. The input `xi` is a design vector of shape $(2k,)$ and the FIM is computed according to (3.17).

Helper function for x_{max} . The function `get_x_max(theta, q=0.95)` chooses the upper bound x_{max} based on the model and the desired saturation level, default is 0.95.

- If `len(theta) == 2` it returns $(q / (1 - q)) * K$
- If `len(theta) == 3` it returns $K * (q / (1 - q))^{**}(1 / h)$.

Design reparation function. PSO and GWO are likely to generate particles that do not satisfy the constraints, e.g.: weights not summing to 1, negative weights, support points x_i outside of $[0, x_{max}]$. To handle this, `repair_design()` has been implemented to clip each x_i back to $[0, x_{max}]$, square weights to make them non-negative, and normalize them by dividing the squared w_i with the sum of squared weights. Then, a valid design array `xi` is returned.

Objective function. `objective_function()` takes a design vector and returns $-\log \det(\mathcal{I}(\xi; \boldsymbol{\theta}))$ as a float if the determinant is larger than 0. Otherwise it returns `1e6` as a penalty value to ensure that the candidate design is treated as poor by the algorithms. `swarm_objective_function()` is the swarm evaluable version actually used in PSO and GWO. It is a wrapper that evaluates `objective_function` at each design vector in the input swarm array, and returns an array containing their objective function values.

Function to compress designs. The raw designs returned by PSO and GWO may contain support points with near-zero weights or several support points that are numerically close to each other. The function `compress_design()` cleans the design by removing weights below a specified tolerance, merging support points that are closer than a given distance tolerance, and renormalizing the remaining weights so that they sum to one.

PSO. Algorithm 1 summarizes the implementation procedure of the PSO function in the module `Loc_D_Opt.py`.

Stopping criterion for PSO:

$$| \Psi_{Gbest} - \Psi_{Cbest} | < \text{tol}_{\text{conv}} \quad \text{or} \quad n_{\text{Iter}} = n_{\text{IterMax}}$$

where tol_{conv} is specified together with other parameters in section 4.3.2 and:

- Ψ_{Gbest} is the global best objective function value found so far, and
- Ψ_{Cbest} is the best objective function value over the swarm for the current iteration.

The inputs are

- `theta`: parameter vector.
- `w, c1, c2`: PSO coefficients.
- `n_particles`: number of particles (candidate designs) in the swarm.
- `n_iter_max`: maximum number of iterations.
- `conv_tol`: threshold for the convergence criterion.

- **seed**: seed for reproducibility of runs.

Algorithm 1. PSO for locally D-optimal designs

- 1: Initialize particles (candidate designs) and velocities
- 2: Evaluate:

$$-\log \det(\mathcal{I}(\xi; \theta))$$
 using `swarm_objective_function()`
- 3: Identify personal best positions, personal best scores, global best position and global best score.
- 4: Set `diff` = ∞ and `n_iter` = 0
- 5: **while** `|diff|` $\geq$ `conv_tol` and `n_iter` < `n_iter_max` **do**
- 6: `n_iter` $\leftarrow$ `n_iter` + 1
- 7: Update particle velocities and particle positions
- 8: Clip particles to the feasible search region
- 9: Re-evaluate objective values
- 10: Set the current best score as the minimum of the current objective values and compute stopping difference `diff`
- 11: Update personal bests and global best
- 12: **end while**
- 13: Repair final global best ξ^* using `repair_design()`
- 14: Compute final $\mathcal{I}(\xi^*; \theta)$ using FIM
- 15: Return ξ^* and results such as $\mathbf{x}^*$, $\mathbf{w}^*$, $\det(\mathcal{I}(\xi^*; \theta))$, $-\log \det(\mathcal{I}(\xi^*; \theta))$, `n_stop`, and `stop_diff`

GWO. Algorithm 2 summarizes the implementation of the GWO function.

Stopping criterion for GWO:

$$|\Psi_{AlphaPrev} - \Psi_{AlphaCurr}| < \text{tol}_{\text{conv}} \quad \text{or} \quad n_{Iter} = n_{IterMax}$$

where:

$\Psi_{AlphaPrev}$ is the previous alpha value of the objective function, and
 $\Psi_{AlphaCurr}$ is the current alpha value of the objective function.

Inputs:

- `theta`, `n_iter_max`, `conv_tol`, `seed`,

- **n_wolves**: number of wolves (candidate designs) in the population.

Algorithm 2. GWO for locally D-optimal designs

1: Initialize wolves (candidate designs)

2: Evaluate:

$$-\log \det(\mathcal{I}(\xi; \boldsymbol{\theta}))$$

using `swarm_objective_function()`

3: Sort wolves by score and identify the three best wolves: α, β, δ

4: Set `diff` = ∞ and `n_iter` = 0

5: **while** `|diff|` $\geq$ `conv_tol` and `n_iter` < `n_iter_max` **do**

6: `n_iter` $\leftarrow$ `n_iter` + 1

7: Update the control parameter:

$$a = 2 - \text{n_iter} \frac{2}{\text{n_iter_max}}$$

8: **for** each wolf $\mathbf{X}_i$ **do**

9: Compute distances to α, β, δ

10: Compute candidate positions X_1, X_2, X_3

11: Update wolf position using

$$\mathbf{X}_i = \frac{\mathbf{X}_1 + \mathbf{X}_2 + \mathbf{X}_3}{3}$$

12: Clip wolf position to the feasible search region

13: **end for**

14: Re-evaluate objective values

15: Update α, β, δ

16: Compute stopping difference

17: **end while**

18: Repair final alpha wolf's position ξ^* using `repair_design()`

19: Compute final $\mathcal{I}(\xi^*; \boldsymbol{\theta})$ using FIM

20: Return ξ^* and results such as $\mathbf{x}^*, \mathbf{w}^*, \det(\mathcal{I}(\xi^*; \boldsymbol{\theta})), -\log \det(\mathcal{I}(\xi^*; \boldsymbol{\theta}))$, final alpha/beta/delta scores, `n_stop`, and `stop_diff`

Further, a function has been implemented to visualize the final design together with the corresponding mean response curve of the nonlinear model. Given the optimized support points and weights for each model and parameter case, $\eta(x; \boldsymbol{\theta})$ is evaluated over a dense grid in the design space and plotted

together with a display of the support points, and corresponding weights as annotations.

The `__main__` block in `Loc_D_Opt.py` is used to run the complete optimization procedure for each parameter vector θ . PSO and GWO are then executed to compute the candidate locally D-optimal designs for both models. After optimization, the resulting designs are cleaned using `compress_design()`; then, the final support points and weights are printed together with results such as the final value of the objective function, and visualized.

4.3.2 Settings for Computing Designs

The following parameter values have been defined inside of the module and are the default arguments for PSO, GWO and `compress_design()`. Unless stated otherwise, these are the values to assume are used by the functions.

- `w = 0.7, c1 = 2.0, c2 = 2.0`
- `n_iter_max = 1000`
- `n_particles = n_wolves = 30`
- `conv_tol = 1e-6`
- `seed = 1`
- `w_tol`: tolerance for removal of weights, weights $< w_tol$ (near-zero weights) are removed from the design. The default is set `w_tol = 1.5e-3`.
- `x_tol`: tolerance for merging. Support points are sorted, and if the distance between x_{i-1} and x_i is smaller than `x_tol`, then x_i is dropped and their weights are merged. The default is set to `x_tol = 1.5e-3`.

4.4 Optimality Check

4.4.1 Implementation Procedure

The implementation of the check of optimality is split into two files: `equivalence.py`, which contains the main functions used to verify the GET

numerically, and `OptimalityCheck.py`, which applies these functions to all model cases and both optimization algorithms. The implementation checks whether all computed values of $\Phi(x, \xi^*)$ are roughly non-positive over a dense grid on the design space, and close to zero at the support points. In particular, `equivalence.py` contains the following:

Sensitivity function and Directional Derivative. The function `d()` computes the sensitivity function as in (3.11) by evaluating each point in the grid. The function `Phi()` computes the directional derivative by subtracting p to the values of `d()` over the grid. This is the quantity used directly in the General Equivalence Theorem check.

Function to check optimality. The function `equivalence_check()` performs the numerical optimality check. It first compresses the design, then creates a grid over $[0, x_{\max}]$, evaluates $\Phi(x, \xi^*)$ over it and at the support points, and returns diagnostic quantities such as

$$\max_x \Phi(x, \xi^*), \quad \max_i |\Phi(x_i^*, \xi^*)|.$$

The design is classified as a SUCCESS if

$$\max_x \Phi(x, \xi^*) \leq \text{tol} \quad \text{and} \quad |\Phi(x_i^*, \xi^*)| \leq \text{tol} \quad \text{for all support points.}$$

Function for vizualization. The function `check_and_plot_equivalence()` is a wrapper around `equivalence_check()`. It runs the same computation, prints the diagnostic results, and plots $\Phi(x, \xi^*)$ over the design space together with the support points of the cleaned design.

Algorithm 3 describes the implementation procedure of `OptimalityCheck.py`.

Algorithm 3. Optimality check for PSO and GWO designs

- 1: Define parameter cases for the Michaelis-Menten model
- 2: Define parameter cases for the Emax model
- 3: Set maximum number of iterations $n_{\text{iter}, \max}$
- 4: **for** each optimization algorithm in $\{\text{PSO}, \text{GWO}\}$ **do**
- 5: **for** each Michaelis-Menten parameter case θ **do**
- 6: Compute candidate design using the optimization algorithm

```

7:      Run:
           check_and_plot_equivalence()

8:      Store the equivalence thm. check results
9:  end for
10:  for each Emax parameter case  $\theta$  do
11:      Compute candidate design using the optimization algorithm
12:      Run:
           check_and_plot_equivalence()

13:      Store the equivalence thm. check results
14:  end for
15: end for

```

4.4.2 Settings for Optimality Check

The following values are set as default and used throughout the entire study.

- **n_grid**: number of grid points used to evaluate $\Phi(x, \xi^*)$ across the design space $[0, x_{\max}]$. The default is set to **n_grid** = 2000.
- **tol**: tolerance used for the equivalence theorem check. The default is set to **tol** = 0.005.

4.5 PSO vs. GWO

4.5.1 Implementation Procedure

The file `PSOvsGWO.py` implements the comparative analysis between PSO and GWO by repeatedly executing both algorithms using different random initializations and storing diagnostic quantities. The implementation uses multiprocessing to execute 1000 independent optimization runs in parallel. The core functions are the following:

Function for one PSO run. The function `one_PSO_run()` performs one independent PSO run. It starts by measuring runtime, then executes `PSO()` to compute a candidate locally D-optimal design. Then, it verifies the optimality of the design using `equivalence_check()`. The function stores quantities such as: runtime, stopping iteration n_{stop} , $\max|\Phi(x, \xi^*)|$ and $\max_i|\Phi(x_i, \xi^*)|$.

It additionally records whether the GET conditions were satisfied according to the specified tolerance.

Function for one GWO run. The function `one_GWO_run()` has the same structure as `one_PSO_run()`, but uses `GWO()` instead of `PSO()`. After computing a candidate D-optimal design, the same equivalence theorem diagnostics are evaluated and stored.

Then, the function `run_parallel_experiment` performs lines 2-8 of the larger scale procedure described in Algorithm 4.

Algorithm 4. Parallel comparison framework for PSO and GWO

- 1: Define parameter cases and number of runs $N = 1000$
- 2: Create process pool with `max_workers` workers
- 3: **for** $i = 1, 2, \dots, N$ **do**
- 4: Submit `one_PSO_run()` to the worker pool
- 5: Submit `one_GWO_run()` to the worker pool
- 6: **end for**
- 7: Collect completed PSO results
- 8: Collect completed GWO results
- 9: Extract collected: runtimes, n_{stop} , $\max|\Phi(x, \xi^*)|$ and $\max_i |\Phi(x_i^*, \xi^*)|$
- 10: Compute summary statistics
- 11: Compute success rates
- 12: Return diagnostic summaries for PSO and GWO

Thus, once all runs have been completed, the relevant quantities are extracted from the stored results and summarized using the established descriptive statistics and success rates. These diagnostics provide a basis for comparing the convergence behavior and computational efficiency of PSO and GWO.

4.6 Response Simulation and Estimation of Parameters

4.6.1 Implementation Procedure

The final step is to compare parameter estimates obtained from the D-optimal design found by PSO with estimates obtained from simpler non-optimal design. The Python file `ResponseSimulation.py` has been created for this purpose. It particularly contains the following functions.

Helper functions.

- Functions to convert weights into exact counts as described in Section 4.2, and to create an array containing the actual design points repeated according to these counts.
- The functions `PSO_design()`, which computes the D-optimal design using `PSO()` and cleans it using `compress_design()`, and `nonoptimal_design()`, which returns one of the equidistant designs with equal weights defined in Section 4.2 as exact designs for a given input "scenario number" (int: 1 or 2).
- A function that generates responses according to

$$y_j = \eta(x_j; \theta) + \varepsilon_j, \quad \varepsilon_j \sim N(0, \sigma^2).$$

- The function `RSS_transformed()`, which evaluates the RSS after transforming the input parameter vector ϕ from the unrestricted space back to the original positive parameter vector: $\theta = \exp(\phi) \mapsto \text{RSS}(\exp(\phi))$.
- The function `MLE()`, which estimates the parameter by minimizing `RSS_transformed()` using the `minimize()` function. The optimizer is initialized at $\phi_0 = \log(\theta_0)$, and the final estimate is recovered as $\hat{\theta} = \exp(\phi)$.

One simulation run. The function `one_simulation_run()` performs one complete simulation for a given design using the functions above. It converts the approximate design into an exact design, simulates responses, creates a perturbed initial θ_0 as in (4.8) and estimates the parameter using `MLE()`.

Repeated simulations. The function `run_simulations()` repeats the simulation procedure for the D-optimal design and the non-optimal designs. For the Michaelis-Menten model, the D-optimal design is compared with both non-optimal scenarios. For the Emax model, the current implementation compares the D-optimal design only with Non-Optimal Scenario 1. Lines 2-10 of Algorithm 5 summarize its implementation.

Algorithm 5. Response simulation and parameter estimation

```

1: Define true  $\theta$ , sample size  $n$ , variance  $\sigma^2$ , and number of simulations  $n_{\text{sims}}$ 
2: Compute the D-optimal design using PSO_design()
3: Define non-optimal design(s) using nonoptimal_design()
4: for each design do
5:   for  $s = 1, 2, \dots, n_{\text{sims}}$  do
6:     Execute one_simulation_run()
7:     Store  $\hat{\theta}$  and minimized RSS
8:   end for
9: end for
10: Return simulation results for all designs

```

After all simulations have been performed, a summary function extracts the estimated parameters ($\hat{V}$, $\hat{K}$ for the MM model, $\hat{V}$, $\hat{K}$, $\hat{h}$ for the Emax model) and computes diagnostic summaries for each across the designs, such as: mean, standard deviation, bias, mean squared error and minimized RSS averaged across simulations.

Functions for plotting histograms and boxplots have also been created and used to visualize the distribution of the parameter estimates across designs.

In the `__main__` block, the full procedure is repeated for each sample size in the list `sample_sizes`. For each n , `run_simulations()` is called, the numerical summaries are printed, and boxplots and histograms are created for comparing the estimated parameters across designs.

4.6.2 Settings for Simulations

The following parameters are used by the code:

- `sigma2`: variance of the noise in the simulated responses. Set to 0.5.

- `n_sims`: number of simulations. Set to 1000.
- `sample_sizes`: sample size n for each study. Set to `[48, 72, 96]`.

Chapter 5

Results

5.1 Computation of Candidate D-Optimal Designs

The tables in this section illustrate the results obtained after running PSO and GWO to yield candidate locally D-optimal designs for each parameter value θ . They include: cleaned support points and weights, minimized $\det [\mathcal{I}(\xi^*; \theta)]$, final objective function value and number of iterations needed to meet stopping criteria.

Table 5.1: PSO results for the Michaelis–Menten model.

$\theta = (V, K)$	x^*	w^*	$\det [\mathcal{I}(\xi^*; \theta)]$	$-\log(\det [\mathcal{I}(\xi^*; \theta)])$	n_{stop}
(3, 0.5)	(0.4525, 9.5)	(0.5, 0.5)	0.4135	0.8831	34
(7, 2)	(1.8094, 38)	(0.5, 0.5)	0.1407	1.9611	43
(5, 8)	(7.2432, 152)	(0.5, 0.5)	0.0045	5.4067	17
(5, 4)	(3.6313, 76)	(0.5, 0.5)	0.0179	4.0204	26

Table 5.2: GWO results for the Michaelis–Menten model.

$\theta = (V, K)$	x^*	w^*	$\det [\mathcal{I}(\xi^*; \theta)]$	$-\log(\det [\mathcal{I}(\xi^*; \theta)])$	n_{stop}
(3, 0.5)	(0.4541, 9.5)	(0.5, 0.5)	0.4135	0.8831	260
(7, 2)	(1.8163, 38)	(0.5, 0.5)	0.1407	1.9611	260
(5, 8)	(7.2650, 152)	(0.5, 0.5)	0.0045	5.4067	260
(5, 4)	(3.6325, 76)	(0.5, 0.5)	0.0179	4.0204	260

Table 5.3: PSO results for the Emax model

$\theta = (V, K, h)$	x^*	w^*	$\det [\mathcal{I}(\xi^*; \theta)]$	$-\log(\det [\mathcal{I}(\xi^*; \theta)])$	n_{stop}
(1, 1, 0.5)	(0.0823, 4.1973, 361)	(1/3, 1/3, 1/3)	1.0531×10^{-4}	9.1586	119
(2, 6, 2)	(3.2189, 8.5926, 26.1534)	(1/3, 1/3, 1/3)	4.6804×10^{-5}	9.9695	101
(3, 0.2, 2)	(0.1071, 0.2865, 0.8718)	(1/3, 1/3, 1/3)	0.2133	1.5453	124
(2.5, 2, 6)	(1.6234, 2.2527, 3.2670)	(1/3, 1/3, 1/3)	0.0010	6.8797	113

Table 5.4: GWO results for the Emax model.

$\theta = (V, K, h)$	x^*	w^*	$\det [\mathcal{I}(\xi^*; \theta)]$	$-\log(\det [\mathcal{I}(\xi^*; \theta)])$	n_{stop}
(1, 1, 0.5)	(0.0819, 4.1293, 361)	(0.3334, 0.3334, 0.3331)	1.0531×10^{-4}	9.1586	934
(2, 6, 2)	(3.2205, 8.5623, 26.1534)	(0.3311, 0.3342, 0.3347)	4.6801×10^{-5}	9.9696	978
(3, 0.2, 2)	(0.1073, 0.2854, 0.8718)	(0.3311, 0.3342, 0.3347)	0.2132	1.5454	978
(2.5, 2, 6)	(1.6237, 2.2540, 3.2670)	(0.3334, 0.3325, 0.3343)	0.0010	6.8808	1000

5.2 Visualization and Optimality Check

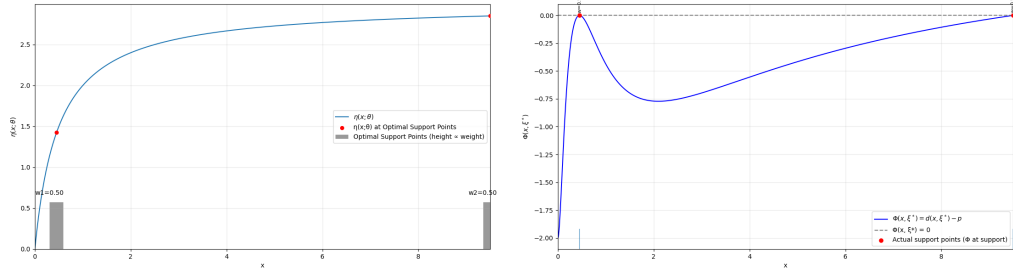
This section contains plots of the mean response function $\eta(x; \theta)$ and the directional derivative $\Phi(x, \xi^*)$ for both nonlinear models across the different parameter cases, each including illustrations of the support points and weights obtained from the algorithms.

The support points are evaluated and displayed as red dots on the curve, while their weights are visualized as gray bars on the x -axis, with height proportional to the weights themselves.

For each case, the tables below include: the value of the upper bound $x_{\max}$ for the considered design space, the values of support points and weights, the value of $\Phi(x_i^*, \xi^*)$ at the support points, the maximum value of $\Phi(x, \xi^*)$ across the grid on the design space, and whether the check of D-optimality of the design was a SUCCESS or a FAILURE.

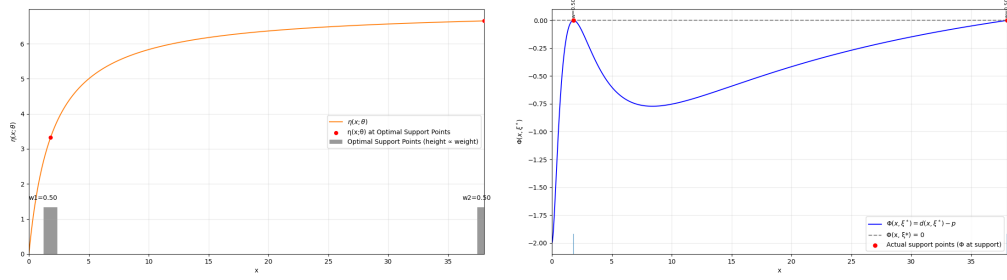
Since $\text{tol} = 0.005$ was chosen, anything ≤ 0.005 is considered ~ 0 . Thus, SUCCESS means:

$$\max_{x \in [0, x_{\max}]} \Phi(x, \xi^*) \leq 0.005 \quad \text{and} \quad |\Phi(x_i^*, \xi^*)| \leq 0.005 \quad \forall x_i^* \in S_{\xi^*}$$



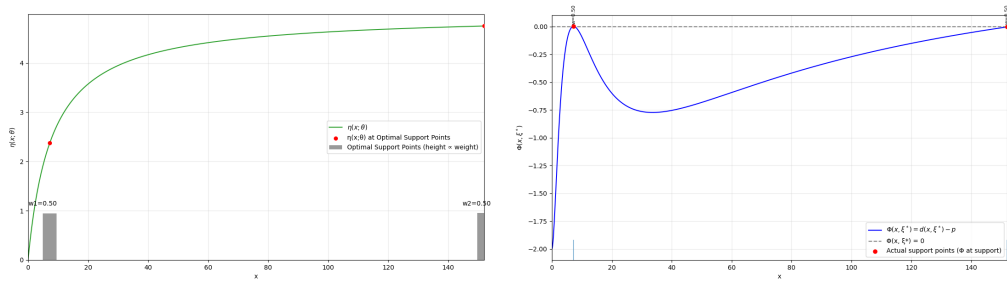
(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.1: PSO D-optimal design for MM model with $\theta = (3, 0.5)$.



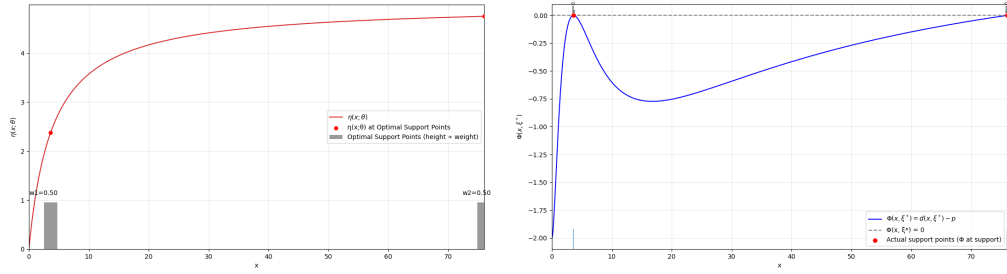
(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.2: PSO D-optimal design for MM model with $\theta = (7, 2)$.



(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.3: PSO D-optimal design for MM model with $\theta = (5, 8)$.

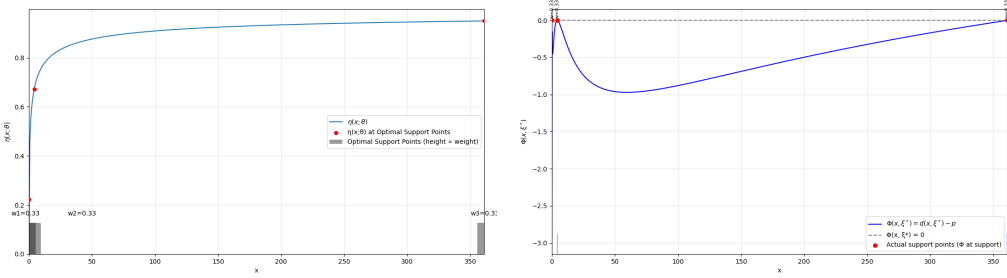


(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.4: PSO D-optimal design for MM model with $\theta = (5, 4)$.

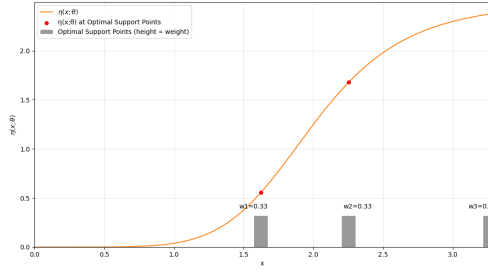
Table 5.5: Equivalence theorem check for the PSO designs in the Michaelis–Menten model.

$\theta = (V, K)$	$x_{\max}$	x_1^*	x_2^*	$\Phi(x_1^*, \xi^*)$	$\Phi(x_2^*, \xi^*)$	$\max_x \Phi(x, \xi^*)$	Status
(3, 0.5)	9.5	0.4525	9.5	-2.38×10^{-10}	-5.96×10^{-10}	-5.96×10^{-10}	SUCCESS
(7, 2)	38	1.8094	38	-6.61×10^{-10}	-7.02×10^{-10}	-7.02×10^{-10}	SUCCESS
(5, 8)	152	7.2432	152	-2.01×10^{-8}	-5.57×10^{-9}	-5.57×10^{-9}	SUCCESS
(5, 4)	76	3.6313	76	-5.05×10^{-9}	-1.80×10^{-9}	1.21×10^{-5}	SUCCESS

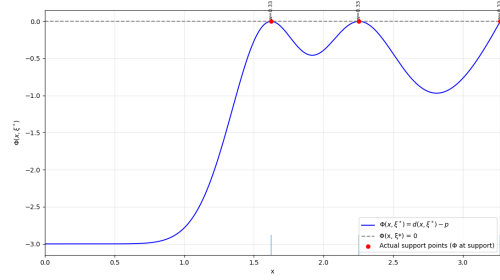


(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.5: PSO D-optimal design for Emax model with $\theta = (1, 1, 0.5)$.

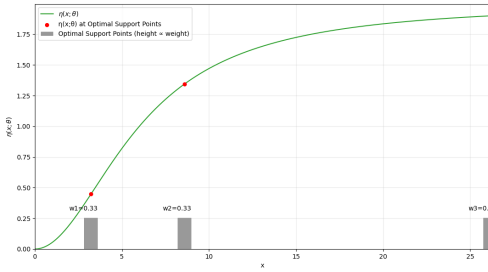


(a) Mean response function $\eta(x; \theta)$.

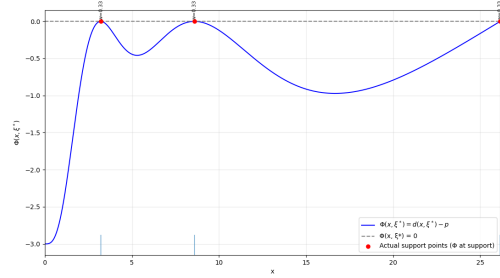


(b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.6: PSO D-optimal design for Emax model with $\theta = (2.5, 2, 6)$.

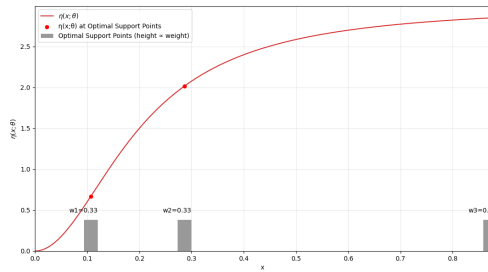


(a) Mean response function $\eta(x; \theta)$.

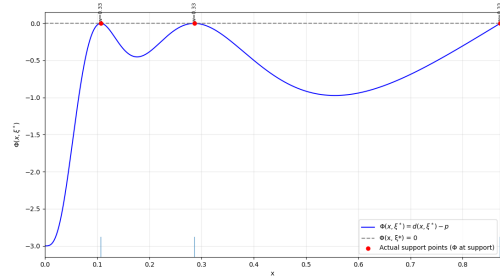


(b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.7: PSO D-optimal design for Emax model with $\theta = (2, 6, 2)$.



(a) Mean response function $\eta(x; \theta)$.



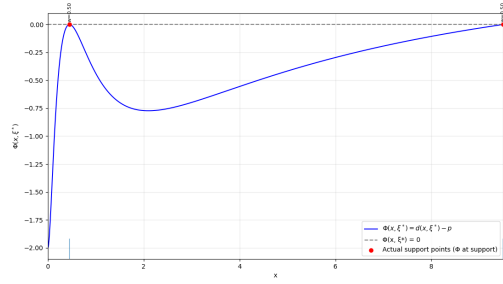
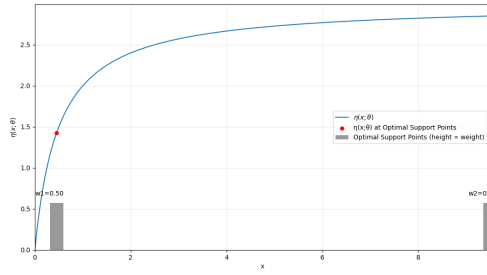
(b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.8: PSO D-optimal design for Emax model with $\theta = (3, 0.2, 2)$.

Table 5.6: Directional derivative values at the support points for the PSO designs in the Emax model.

$\theta = (V, K, h)$	$x_{\max}$	x_1^*	x_2^*	x_3^*	$\Phi(x_1^*, \xi^*)$	$\Phi(x_2^*, \xi^*)$	$\Phi(x_3^*, \xi^*)$
(1, 1, 0.5)	361	0.0823	4.1973	361	-6.66×10^{-9}	-7.32×10^{-8}	-4.34×10^{-8}
(2.5, 2, 6)	3.2670	1.6234	2.2527	3.2670	-5.78×10^{-8}	-4.38×10^{-8}	-1.02×10^{-8}
(2, 6, 2)	26.1534	3.2189	8.5926	26.1534	-1.24×10^{-8}	-4.78×10^{-8}	-2.65×10^{-8}
(3, 0.2, 2)	0.8718	0.1071	0.2865	0.8718	-4.60×10^{-9}	-3.73×10^{-9}	-2.43×10^{-9}

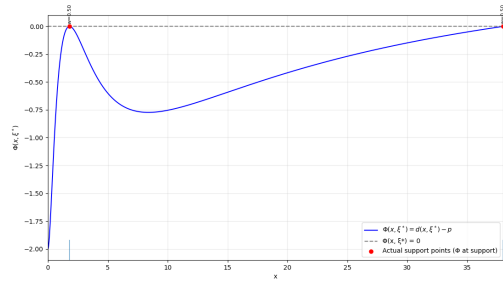
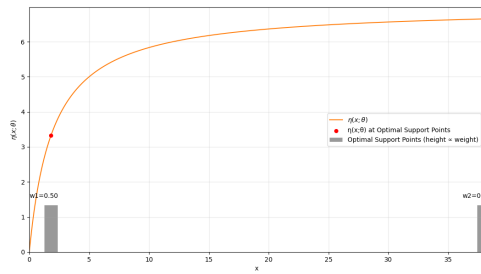
$\theta = (V, K, h)$	$\max_x \Phi(x, \xi^*)$	Status
(1, 1, 0.5)	-4.34×10^{-8}	SUCCESS
(2.5, 2, 6)	3.26×10^{-5}	SUCCESS
(2, 6, 2)	4.16×10^{-5}	SUCCESS
(3, 0.2, 2)	1.19×10^{-5}	SUCCESS



(a) Mean response function $\eta(x; \theta)$.

(b) Directional derivative $\Phi(x, \xi^*)$.

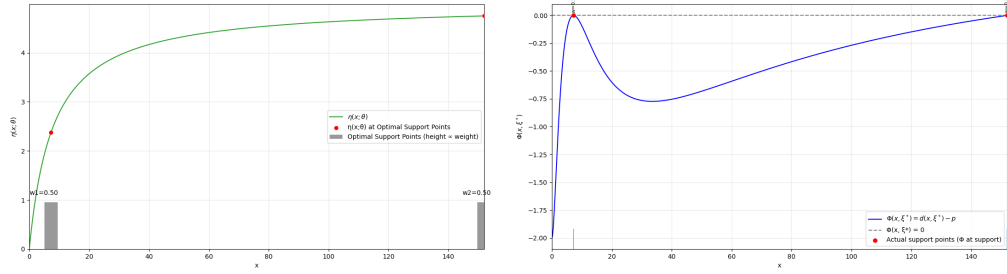
Figure 5.9: GWO D-optimal design for MM model with $\theta = (3, 0.5)$.



(a) Mean response function $\eta(x; \theta)$.

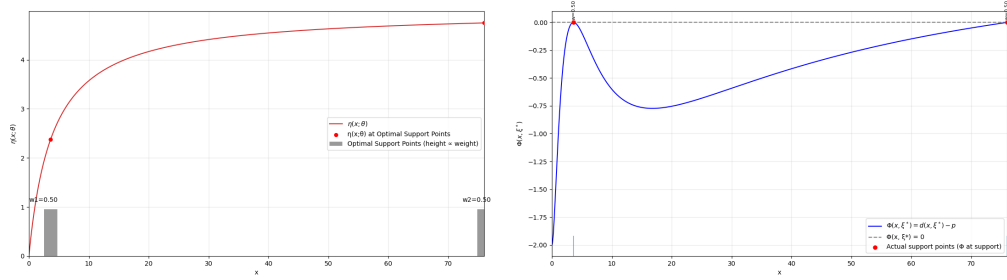
(b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.10: GWO D-optimal design for MM model with $\theta = (7, 2)$.



(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.11: GWO D-optimal design for MM model with $\theta = (5, 8)$.

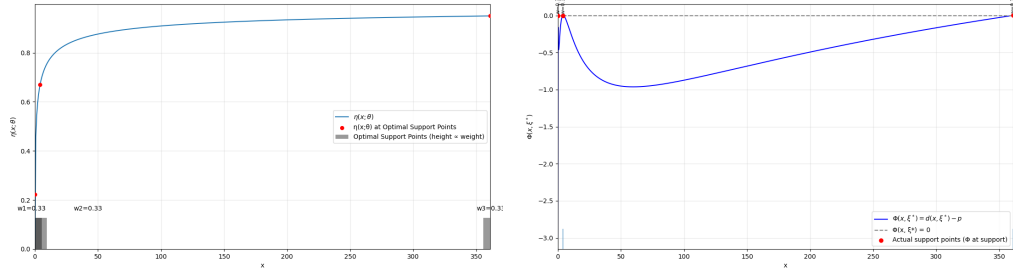


(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.12: GWO D-optimal design for MM model with $\theta = (5, 4)$.

Table 5.7: Equivalence theorem check for the GWO designs in the Michaelis–Menten model.

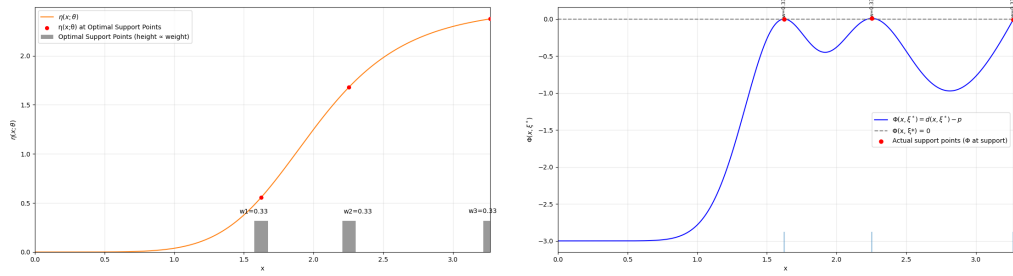
$\theta = (V, K)$	$x_{\max}$	x_1^*	x_2^*	$\Phi(x_1^*, \xi^*)$	$\Phi(x_2^*, \xi^*)$	$\max_x \Phi(x, \xi^*)$	Status
(3, 0.5)	9.5	0.4541	9.5	-2.38×10^{-10}	-5.96×10^{-10}	1.53×10^{-5}	SUCCESS
(7, 2)	38	1.8163	38	-6.61×10^{-10}	-7.03×10^{-10}	1.53×10^{-5}	SUCCESS
(5, 8)	152	7.2650	152	-2.01×10^{-8}	-5.59×10^{-9}	1.53×10^{-5}	SUCCESS
(5, 4)	76	3.6325	76	-5.05×10^{-9}	-1.80×10^{-9}	1.53×10^{-5}	SUCCESS



(a) Mean response function $\eta(x; \theta)$.

(b) Directional derivative $\Phi(x, \xi^*)$.

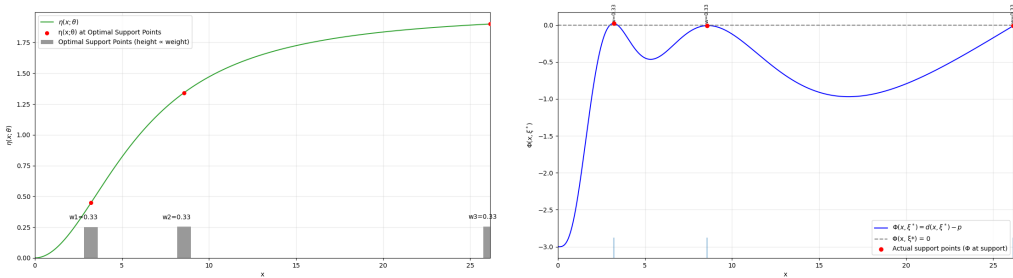
Figure 5.13: GWO D-optimal design for Emax model with $\theta = (1, 1, 0.5)$.



(a) Mean response function $\eta(x; \theta)$.

(b) Directional derivative $\Phi(x, \xi^*)$.

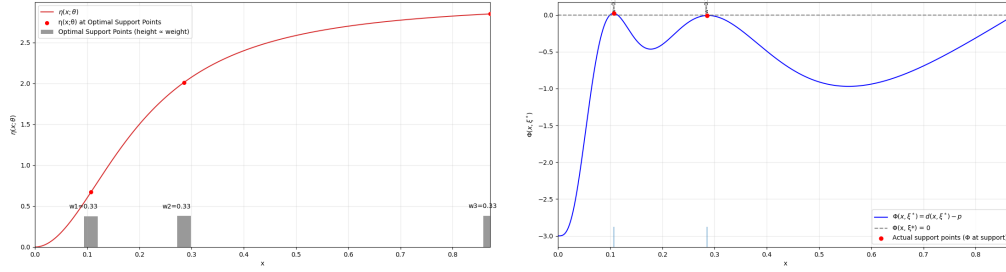
Figure 5.14: GWO D-optimal design for Emax model with $\theta = (2.5, 2, 6)$.



(a) Mean response function $\eta(x; \theta)$.

(b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.15: GWO D-optimal design for Emax model with $\theta = (2, 6, 2)$.



(a) Mean response function $\eta(x; \theta)$. (b) Directional derivative $\Phi(x, \xi^*)$.

Figure 5.16: GWO D-optimal design for Emax model with $\theta = (3, 0.2, 2)$.

Table 5.8: Equivalence theorem check for the GWO designs in the Emax model.

$\theta = (V, K, h)$	$x_{\max}$	x_1^*	x_2^*	x_3^*	$\Phi(x_1^*, \xi^*)$	$\Phi(x_2^*, \xi^*)$	$\Phi(x_3^*, \xi^*)$
(1, 1, 0.5)	361	0.0819	4.1293	361	-9.25×10^{-4}	-9.25×10^{-4}	1.85×10^{-3}
(2.5, 2, 6)	3.2670	1.6237	2.2540	3.2670	-3.79×10^{-4}	8.84×10^{-3}	-8.41×10^{-3}
(2, 6, 2)	26.1534	3.2205	8.5623	26.1534	2.03×10^{-2}	-8.10×10^{-3}	-1.20×10^{-2}
(3, 0.2, 2)	0.8718	0.1073	0.2854	0.8718	2.03×10^{-2}	-8.10×10^{-3}	-1.20×10^{-2}

$\theta = (V, K, h)$	$\max_x \Phi(x, \xi^*)$	Status
(1, 1, 0.5)	1.85×10^{-3}	SUCCESS
(2.5, 2, 6)	8.84×10^{-3}	FAILURE
(2, 6, 2)	2.05×10^{-2}	FAILURE
(3, 0.2, 2)	2.05×10^{-2}	FAILURE

5.3 PSO vs. GWO

The following subsection presents the results obtained from the comparative study described in Section 4.5, where PSO and GWO were applied to construct locally D-optimal designs for the Michaelis-Menten and Emax models repeatedly. For each parameter case, both algorithms were executed 1000 times using different random initializations. The tables report summary statistics for: the stopping iteration n_{stop} , the runtime of the algorithm, the quantity $\max_i (|\Phi(x_i, \xi^*)|)$ evaluated at the support points of the obtained design, which measures how closely the GET condition is satisfied at the support points, and the proportion of successful runs, defined as runs for which the GET criteria (4.4) and (4.3) were satisfied within $\text{tol} = 0.005$.

Table 5.9: Comparative diagnostics for PSO and GWO for the Michaelis–Menten model with $\theta = [3, 0.5]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	0.9190	12.8023	max	111	992
Q3	0.2823	8.1809	Q3	42	883
median	0.2303	6.9752	median	35	740
Q1	0.1898	5.1537	Q1	30	533.75
min	0.0803	0.0755	min	13	6
mean	0.2427	6.5234	mean	36.950	685.046

Max equivalence-thm. deviation at support points: $\max_i (\Phi(x_i, \xi^*))$			Empirical success rates		
Diagnostic	PSO	GWO	Check	PSO	GWO
max	0.0718	0.2842	Full GET	95.80%	79.30%
Q3	5.96×10^{-10}	0.0027	Support points only	95.80%	80.10%
median	5.96×10^{-10}	5.96×10^{-10}			
Q1	5.96×10^{-10}	5.96×10^{-10}			
min	5.95×10^{-10}	5.86×10^{-10}			
mean	0.0008	0.0064			

Table 5.10: Comparative diagnostics for PSO and GWO for the Michaelis–Menten model with $\theta = [7, 2]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	0.9206	13.6119	max	109	992
Q3	0.2700	8.3454	Q3	40	883
median	0.2270	7.0715	median	34	740
Q1	0.1864	5.1688	Q1	28	533.75
min	0.0585	0.0821	min	7	6
mean	0.2345	6.6400	mean	35.091	685.046

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$			Empirical success rates		
Diagnostic	PSO	GWO	Check	PSO	GWO
max	0.5626	0.2842	Full GET	95.80%	79.30%
Q3	7.02×10^{-10}	0.0027	Support points only	95.80%	80.10%
median	7.02×10^{-10}	7.02×10^{-10}			
Q1	7.02×10^{-10}	7.02×10^{-10}			
min	6.99×10^{-10}	6.82×10^{-10}			
mean	0.0017	0.0064			

Table 5.11: Comparative diagnostics for PSO and GWO for the Michaelis–Menten model with $\theta = [5, 8]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	1.0125	11.3552	max	156	992
Q3	0.2814	7.9432	Q3	41	883
median	0.2333	6.7295	median	34	740
Q1	0.1882	4.8656	Q1	28	533.75
min	0.0348	0.0523	min	8	6
mean	0.2444	6.3043	mean	35.591	685.046

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$		
Diagnostic	PSO	GWO
max	2.0000	0.2842
Q3	2.01×10^{-8}	0.0027
median	2.01×10^{-8}	2.01×10^{-8}
Q1	2.01×10^{-8}	2.01×10^{-8}
min	2.01×10^{-8}	2.01×10^{-8}
mean	0.0029	0.0064

Empirical success rates		
Check	PSO	GWO
Full GET	95.40%	79.30%
Support points only	95.60%	80.10%

Table 5.12: Comparative diagnostics for PSO and GWO for the Michaelis–Menten model with $\theta = [5, 4]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	1.2495	13.2031	max	118	992
Q3	0.2888	8.0146	Q3	41	883
median	0.2373	6.6813	median	34	740
Q1	0.1942	4.7705	Q1	28	533.75
min	0.0781	0.0627	min	9	6
mean	0.2491	6.3500	mean	35.35	685.046

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$		
Diagnostic	PSO	GWO
max	0.6487	0.2842
Q3	5.05×10^{-9}	0.0027
median	5.05×10^{-9}	5.05×10^{-9}
Q1	5.05×10^{-9}	5.05×10^{-9}
min	5.05×10^{-9}	5.05×10^{-9}
mean	0.0018	0.0064

Empirical success rates		
Check	PSO	GWO
Full GET	95.40%	79.30%
Support points only	95.40%	80.10%

Table 5.13: Comparative diagnostics for PSO and GWO for the Emax model with $\theta = [1, 1, 0.5]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	2.4041	23.6264	max	258	998
Q3	1.1767	13.5867	Q3	116	979
median	0.9933	12.4383	median	99	956
Q1	0.8397	11.4597	Q1	85	900
min	0.3804	0.6425	min	40	54
mean	1.0329	12.3269	mean	102.679	904.332

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$			Empirical success rates		
Diagnostic	PSO	GWO	Check	PSO	GWO
max	2.9998	1.0849	Full GET	58.50%	55.00%
Q3	0.0109	0.0140	Support points only	58.60%	57.00%
median	0.0014	0.0031			
Q1	7.32×10^{-8}	7.32×10^{-8}			
min	7.25×10^{-8}	7.20×10^{-8}			
mean	0.1652	0.0195			

Table 5.14: Comparative diagnostics for PSO and GWO for the Emax model with $\theta = [2.5, 2, 6]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	2.5389	21.9430	max	271	1000
Q3	1.3657	14.5922	Q3	125.25	1000
median	1.1383	13.5898	median	106	1000
Q1	0.9235	12.7924	Q1	86	997
min	0.3982	5.0277	min	38	323
mean	1.1721	13.8700	mean	108.387	994.343

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$			Empirical success rates		
Diagnostic	PSO	GWO	Check	PSO	GWO
max	1.9997	3.1231	Full GET	59.60%	27.50%
Q3	0.0109	0.0134	Support points only	59.70%	27.50%
median	0.0017	0.0080			
Q1	5.77×10^{-8}	0.0046			
min	5.69×10^{-8}	0.0002			
mean	0.0274	0.1581			

Table 5.15: Comparative diagnostics for PSO and GWO for the Emax model with $\theta = [2, 6, 2]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	2.5913	17.1102	max	285	1000
Q3	1.1340	12.5338	Q3	118	998
median	0.9414	12.0685	median	98	995
Q1	0.7704	11.5796	Q1	80	986
min	0.0197	2.8199	min	1	207
mean	0.9767	11.8569	mean	101.793	981.243

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$			Empirical success rates		
Diagnostic	PSO	GWO	Check	PSO	GWO
max	2.4937	1.5010	Full GET	58.80%	50.80%
Q3	0.0102	0.0135	Support points only	58.90%	51.30%
median	0.0013	0.0047			
Q1	4.78×10^{-8}	0.0023			
min	4.78×10^{-8}	4.78×10^{-8}			
mean	0.0183	0.0192			

Table 5.16: Comparative diagnostics for PSO and GWO for the Emax model with $\theta = [3, 0.2, 2]$ over 1000 independent runs.

Runtime			Stopping iteration n_{stop}		
Diagnostic	PSO	GWO	Diagnostic	PSO	GWO
max	2.1784	15.8459	max	291	1000
Q3	0.9670	10.8673	Q3	126	998
median	0.8001	9.9737	median	104.5	995
Q1	0.6736	9.4597	Q1	87	986
min	0.2753	2.4165	min	35	207
mean	0.8400	10.3293	mean	109.58	981.2400

Max equivalence-thm. deviation at support points: $\max_i (d(x_i, \xi^*))$			Empirical success rates		
Diagnostic	PSO	GWO	Check	PSO	GWO
max	1.4969	1.5010	Full GET	57.30%	50.70%
Q3	0.0109	0.0135	Support points only	57.40%	51.20%
median	0.0026	0.0047			
Q1	4.61×10^{-9}	0.0023			
min	4.55×10^{-9}	4.29×10^{-9}			
mean	0.0125	0.0192			

5.4 Response Simulation and Estimation of Parameters

This subsection presents the simulation study described in Section 4.6, where responses were generated under both locally D-optimal and non-optimal experimental designs for the Michaelis-Menten and Emax models.

- **Non-Optimal Design 1**

$$\text{Support points: } \mathbf{x} = \left[0, \frac{x_{max}}{2}, x_{max} \right]$$

$$\text{Weights: } \mathbf{w} = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right]$$

- **Non-Optimal Design 2**

$$\text{Support points: } \mathbf{x} = \left[0, \frac{x_{max}}{4}, \frac{3 \cdot x_{max}}{4}, x_{max} \right]$$

$$\text{Weights: } \mathbf{w} = \left[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4} \right]$$

For each design, repeated datasets were simulated and the parameters were estimated using nonlinear least squares. The resulting parameter estimates are summarized using boxplots, histograms and different diagnostics.

Table 5.17: Simulation diagnostics for the Michaelis-Menten model with true parameter values $V = 3$, $K = 0.5$ and sample size $n = 48$.

Diagnostic	D-Optimal	Non-Optimal 1	Non-Optimal 2
Support points	[0.4525, 9.5]	[0, 4.75, 9.5]	[0, 2.375, 7.125, 9.5]
Weights	[0.5, 0.5]	[0.3333, 0.3333, 0.3333]	[0.25, 0.25, 0.25, 0.25]
Mean($\hat{V}$)	3.0079	3.1139	3.0241
SD($\hat{V}$)	0.1655	0.4067	0.2524
Bias($\hat{V}$)	0.0079	0.1139	0.0241
MSE($\hat{V}$)	0.0275	0.1783	0.0643
Mean($\hat{K}$)	0.5139	0.7922	0.5556
SD($\hat{K}$)	0.1241	0.9513	0.4128
Bias($\hat{K}$)	0.0139	0.2922	0.0556
MSE($\hat{K}$)	0.0156	0.9903	0.1735
Average RSS	22.9547	23.1089	23.0980

Table 5.18: Simulation diagnostics for the Michaelis-Menten model with true parameter values $V = 3$, $K = 0.5$ and sample size $n = 72$.

Diagnostic	D-Optimal	Non-Optimal 1	Non-Optimal 2
Support points	(0.4525, 9.5)	(0, 4.75, 9.5)	(0, 2.375, 7.125, 9.5)
Weights	(0.5, 0.5)	(0.3333, 0.3333, 0.3333)	(0.25, 0.25, 0.25, 0.25)
Mean($\hat{V}$)	3.0022	3.0640	3.0172
SD($\hat{V}$)	0.1395	0.3174	0.2062
Bias($\hat{V}$)	0.0022	0.0640	0.0172
MSE($\hat{V}$)	0.0195	0.1048	0.0428
Mean($\hat{K}$)	0.5091	0.6575	0.5373
SD($\hat{K}$)	0.1031	0.7096	0.3420
Bias($\hat{K}$)	0.0091	0.1575	0.0373
MSE($\hat{K}$)	0.0107	0.5283	0.1184
Average RSS	34.7743	35.2660	34.9063

Table 5.19: Simulation diagnostics for the Michaelis-Menten model with true parameter values $V = 3$, $K = 0.5$ and sample size $n = 96$.

Diagnostic	D-Optimal	Non-Optimal 1	Non-Optimal 2
Support points	(0.4525, 9.5)	(0, 4.75, 9.5)	(0, 2.375, 7.125, 9.5)
Weights	(0.5, 0.5)	(0.3333, 0.3333, 0.3333)	(0.25, 0.25, 0.25, 0.25)
Mean($\hat{V}$)	2.9950	3.0570	3.0123
SD($\hat{V}$)	0.1198	0.2948	0.1846
Bias($\hat{V}$)	-0.0050	0.0570	0.0123
MSE($\hat{V}$)	0.0144	0.0901	0.0342
Mean($\hat{K}$)	0.5017	0.6449	0.5191
SD($\hat{K}$)	0.0879	0.6576	0.3008
Bias($\hat{K}$)	0.0017	0.1449	0.0191
MSE($\hat{K}$)	0.0077	0.4534	0.0908
Average RSS	46.9153	47.2165	46.7096

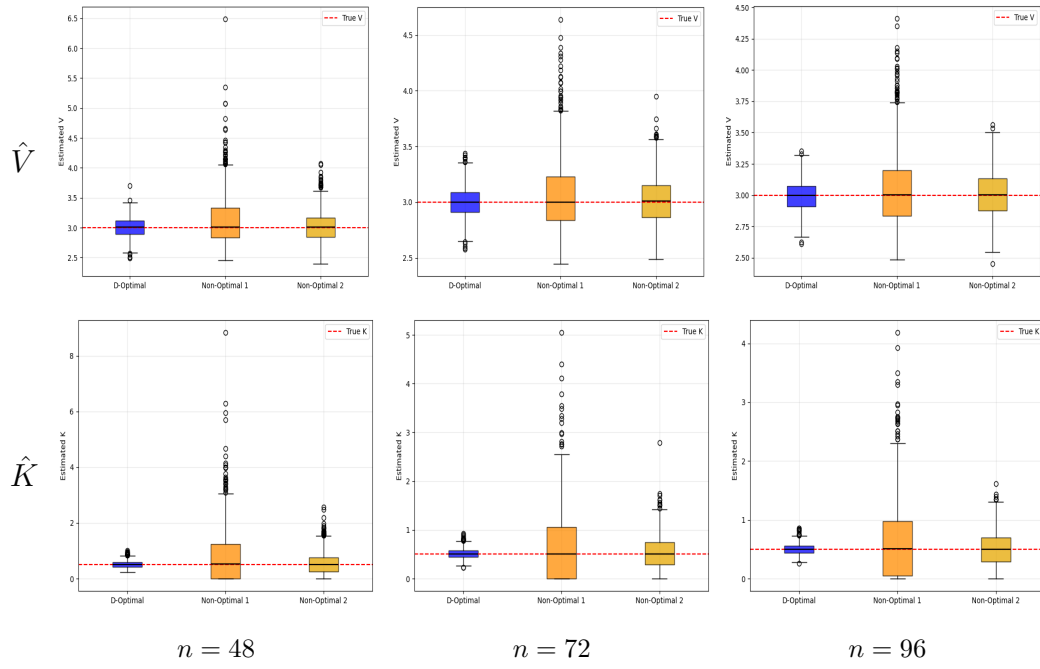


Figure 5.17: Boxplots of the parameter estimates $\hat{V}$ and $\hat{K}$ for the Michaelis–Menten model under the D-optimal design and the two non-optimal designs, for sample sizes $n = 48, 72, 96$.

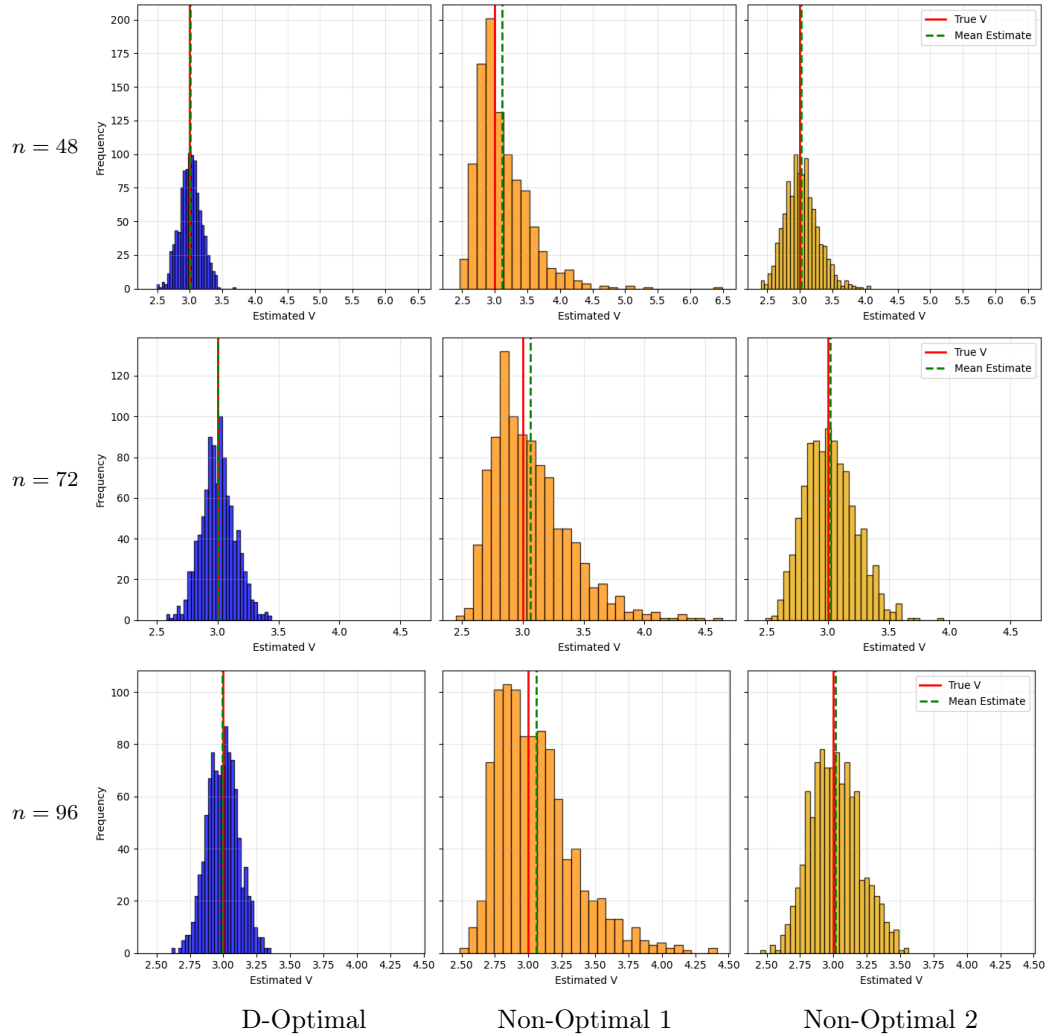


Figure 5.18: Histograms of the estimated parameter $\hat{V}$ for the Michaelis–Menten model under the D-optimal design and the two non-optimal designs, for sample sizes $n = 48, 72, 96$. The red dashed line denotes the true parameter value, while the green dashed line denotes the sample mean of the estimates.

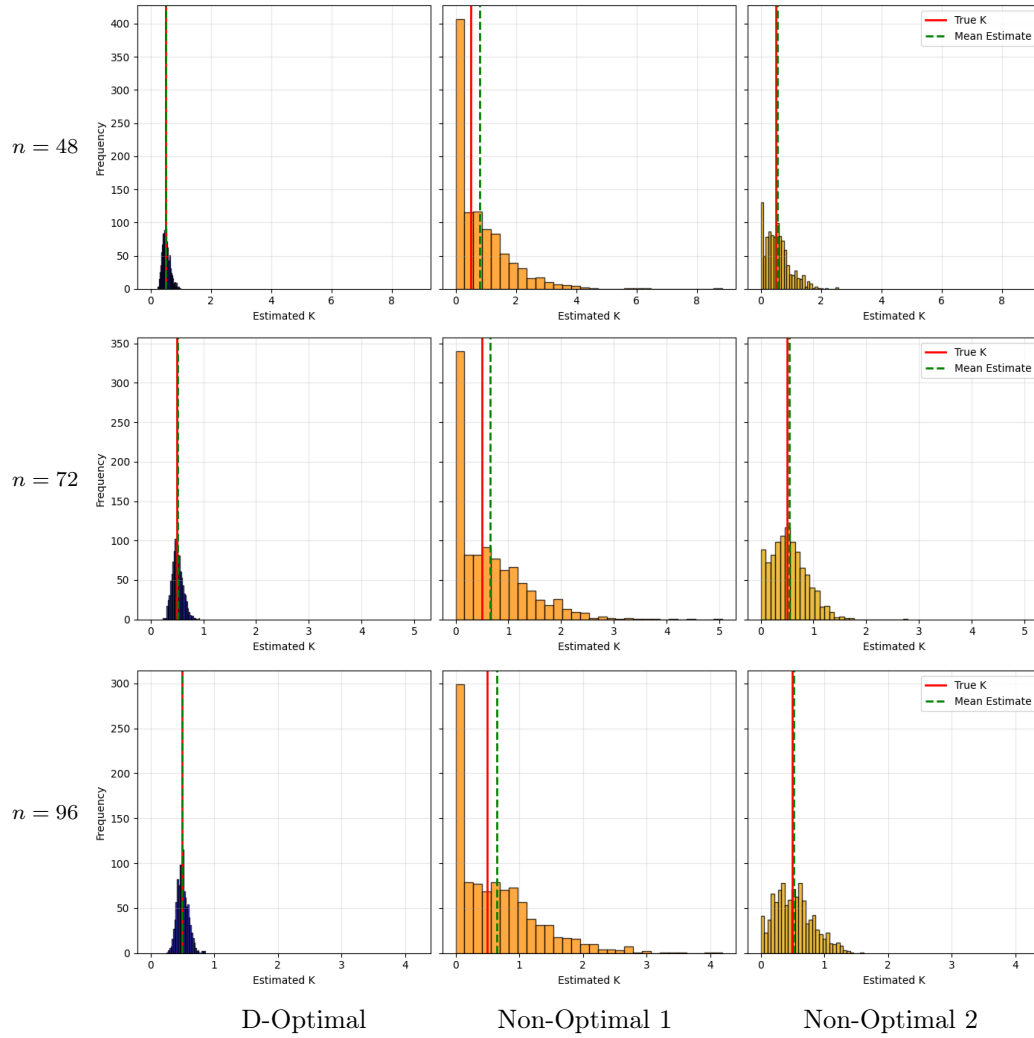


Figure 5.19: Histograms of the estimated parameter $\hat{K}$ for the Michaelis–Menten model under the D-optimal design and the two non-optimal designs, for sample sizes $n = 48, 72, 96$. The red dashed line denotes the true parameter value, while the green dashed line denotes the sample mean of the estimates.

Table 5.20: Simulation diagnostics for the Emax model with true parameter values $V = 3$, $K = 0.2$, $h = 2$.

$n = 48$			$n = 72$		
Diag.	D-Opt.	Non-Opt. 1	Diag.	D-Opt.	Non-Opt. 1
Support pts.	(0.107, 0.287, 0.872)	(0, 0.436, 0.872)	Support pts.	(0.107, 0.287, 0.872)	(0, 0.436, 0.872)
Weights	(0.333, 0.333, 0.333)	(0.333, 0.333, 0.333)	Weights	(0.333, 0.333, 0.333)	(0.333, 0.333, 0.333)
Mean($\hat{V}$)	13.14	3.087	Mean($\hat{V}$)	3.051	3.050
SD($\hat{V}$)	316.56	0.372	SD($\hat{V}$)	0.293	0.296
Bias($\hat{V}$)	10.14	0.087	Bias($\hat{V}$)	0.051	0.050
MSE($\hat{V}$)	1.00×10^5	0.146	MSE($\hat{V}$)	0.089	0.090
Mean($\hat{K}$)	1.49×10^4	0.203	Mean($\hat{K}$)	0.208	0.200
SD($\hat{K}$)	4.72×10^5	0.080	SD($\hat{K}$)	0.040	0.065
Bias($\hat{K}$)	1.49×10^4	0.003	Bias($\hat{K}$)	0.008	-0.0001
MSE($\hat{K}$)	2.23×10^{11}	0.007	MSE($\hat{K}$)	0.002	0.004
Mean($\hat{h}$)	2.059	2.245	Mean($\hat{h}$)	2.044	2.092
SD($\hat{h}$)	0.609	1.074	SD($\hat{h}$)	0.446	0.688
Bias($\hat{h}$)	0.059	0.245	Bias($\hat{h}$)	0.044	0.092
MSE($\hat{h}$)	0.374	1.213	MSE($\hat{h}$)	0.201	0.482
Avg. RSS	22.35	23.01	Avg. RSS	34.33	35.27

$n = 96$		
Diag.	D-Opt.	Non-Opt. 1
Support pts.	(0.107, 0.287, 0.872)	(0, 0.436, 0.872)
Weights	(0.333, 0.333, 0.333)	(0.333, 0.333, 0.333)
Mean($\hat{V}$)	3.036	3.043
SD($\hat{V}$)	0.279	0.246
Bias($\hat{V}$)	0.036	0.043
MSE($\hat{V}$)	0.079	0.062
Mean($\hat{K}$)	0.206	0.200
SD($\hat{K}$)	0.049	0.052
Bias($\hat{K}$)	0.006	-0.001
MSE($\hat{K}$)	0.002	0.003
Mean($\hat{h}$)	2.049	2.060
SD($\hat{h}$)	0.394	0.629
Bias($\hat{h}$)	0.049	0.060
MSE($\hat{h}$)	0.157	0.399
Avg. RSS	46.44	47.21

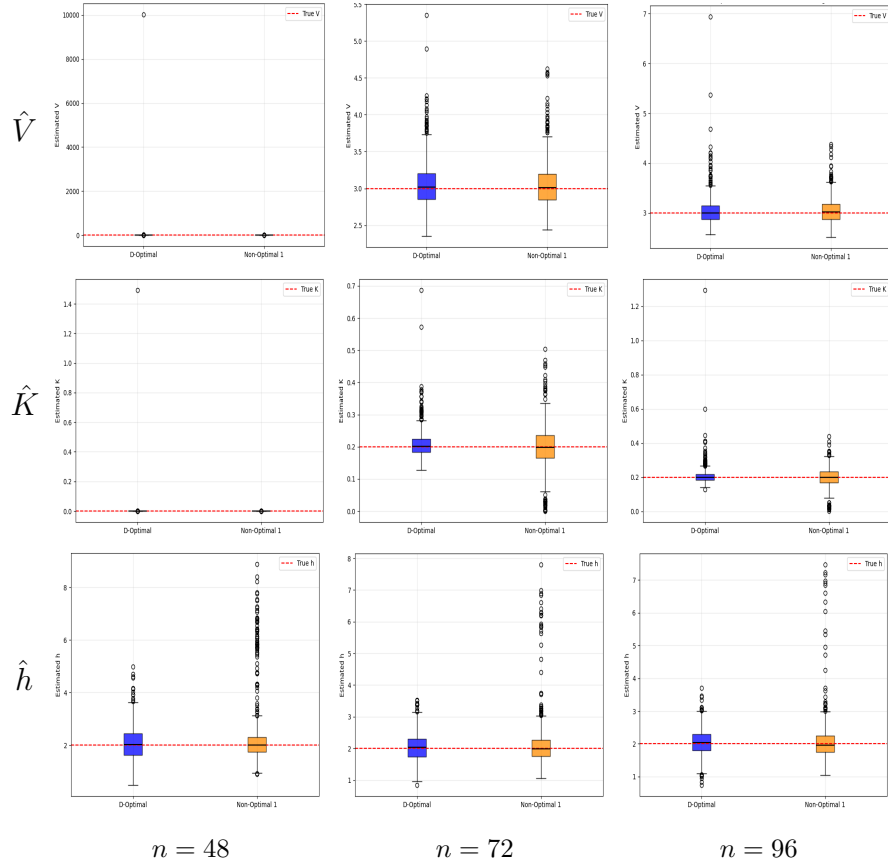


Figure 5.20: Boxplots of the parameter estimates $\hat{V}$, $\hat{K}$, and $\hat{h}$ for the Emax model under the D-optimal design and the non-optimal design, for sample sizes $n = 48, 72, 96$.

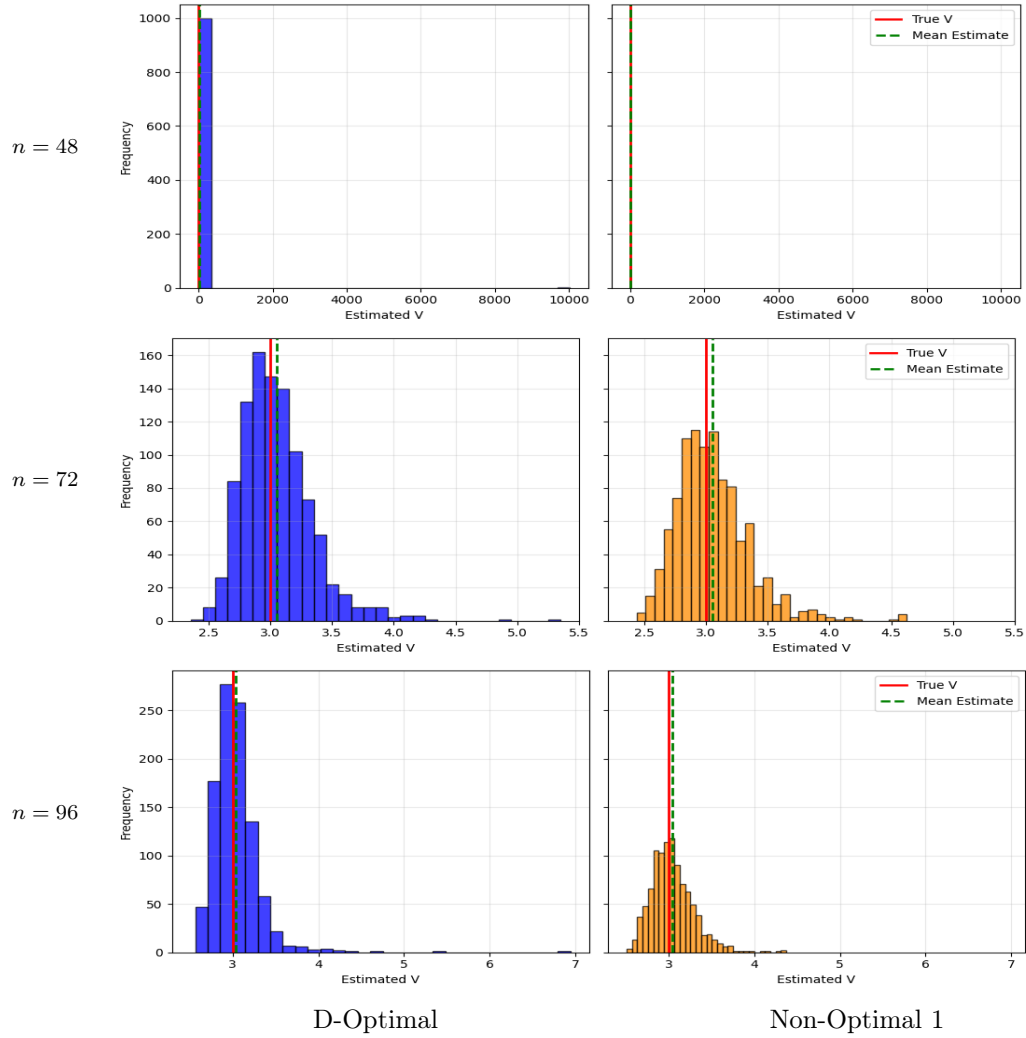


Figure 5.21: Histograms of the estimated parameter $\hat{V}$ for the Emax model under the D-optimal design and the non-optimal design, for sample sizes $n = 48, 72, 96$. The red dashed line denotes the true parameter value, while the green dashed line denotes the sample mean of the estimates.

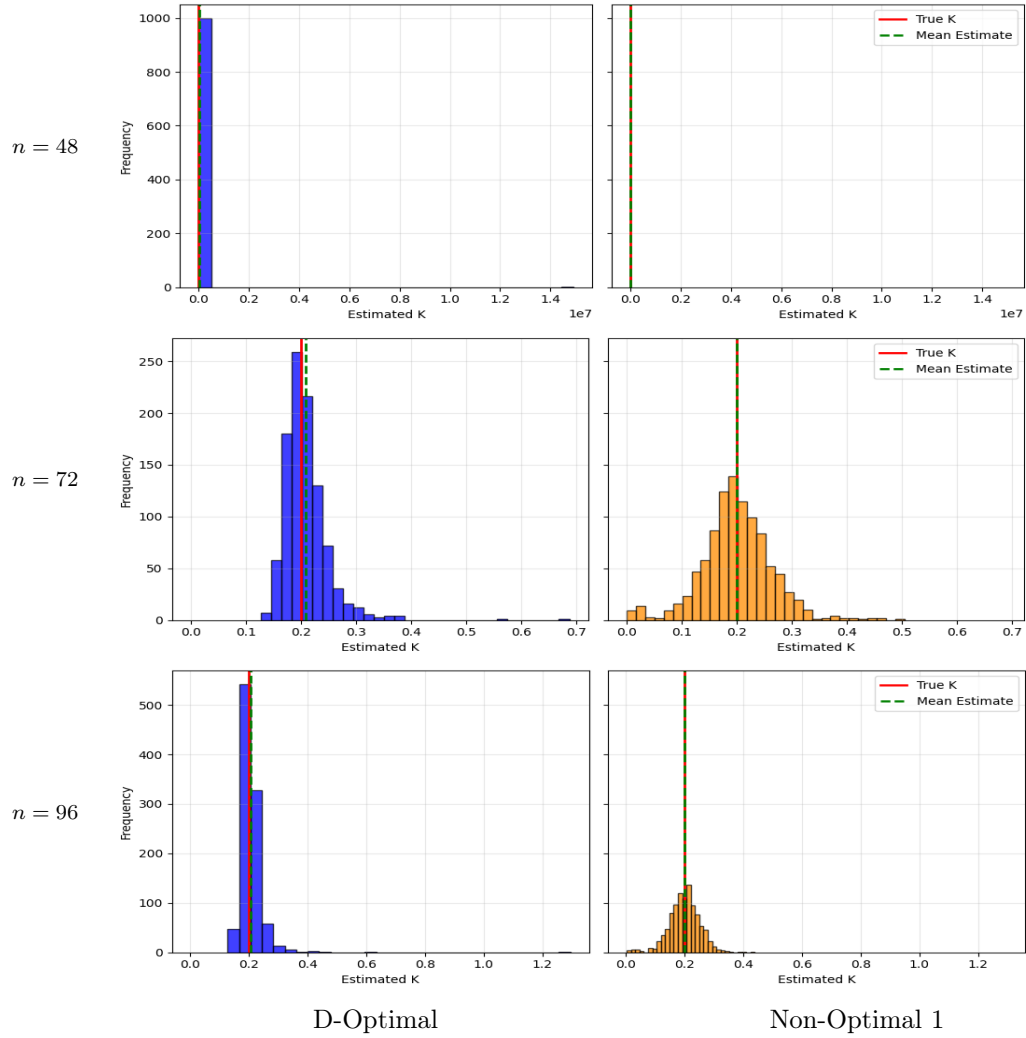


Figure 5.22: Histograms of the estimated parameter $\hat{K}$ for the Emax model under the D-optimal design and the non-optimal design, for sample sizes $n = 48, 72, 96$. The red dashed line denotes the true parameter value, while the green dashed line denotes the sample mean of the estimates.

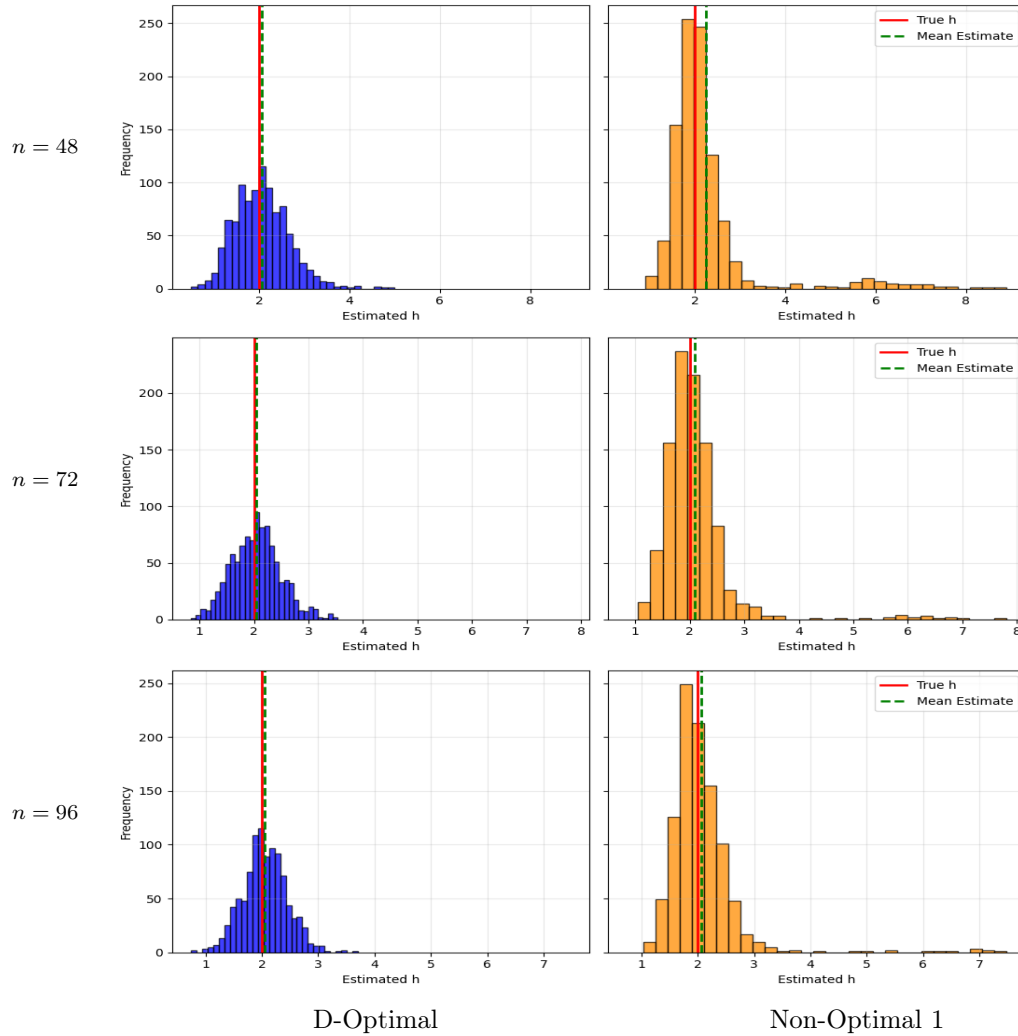


Figure 5.23: Histograms of the estimated parameter $\hat{h}$ for the Emax model under the D-optimal design and the non-optimal design, for sample sizes $n = 48, 72, 96$. The red dashed line denotes the true parameter value, while the green dashed line denotes the sample mean of the estimates.

Chapter 6

Discussion

6.1 Obtained Designs and Local D-Optimality

The obtained designs exhibited several consistent patterns across both the Michaelis-Menten and Emax models. In general, the algorithms seem to try to maximize information by positioning the support points in regions where the regression function changes most rapidly, while also allocating observations at the upper bound of the design space.

For the Michaelis-Menten model, the cleaned designs consistently produced two support points with approximately equal weights. The first support point was typically located near the region where the curve begins to bend, seemingly corresponding to where the curve transitions from initially having an approximately linear behavior to approaching saturation. Points in this region certainly capture information about the parameter K , since they have values close to it and capture function values close to half of the maximum response.

The second support point was consistently located at the largest value in the design space, $x_{\max}$, just about where the response approaches the asymptotic maximum V , intuitively capturing information about the saturation level of the model. The fact that the algorithms tend to place the 2nd support point at the upper bound of the design space may also indicate that a larger value of $x_{\max}$ could potentially allow the algorithms to place support points further into the saturation region. Even so, the obtained designs still show a stable

structure across all choices of parameters.

For the Emax model, the resulting designs generally consisted of three support points with nearly uniform weights after cleaning. In contrast to the Michaelis-Menten model, the Emax designs included an additional intermediate support point consistently located where the curve changes most rapidly, which is reasonable since the Emax model contains the additional shape parameter h , which controls the steepness of the response curve.

Another thing worth noting is how, in nearly all cases, the nonzero weights obtained after compressing and cleaning were approximately uniform. This likely suggests that the algorithms converged to positions in the design space that are balanced in terms of information.

The equivalence theorem diagnostics further supported the local D-optimality of the obtained designs. In the plots of the directional derivative function $\Phi(x, \xi^*)$, the support points were located in regions where the function attained values close to zero, while the remaining values over the design space stayed non-positive. Therefore, the directional derivative plots visually agreed with the conditions of the General Equivalence Theorem.

However, more significant deviations from zero were observed in certain three-parameter Emax cases. In particular, the designs obtained from GWO frequently failed to satisfy the conditions of the optimality check, under a somewhat strict tolerance. Nevertheless, taken together, the diagnostics and the overall behavior of the directional derivative strongly indicate nearly locally D-optimal solutions from both metaheuristic approaches.

6.2 Comparison Between PSO and GWO

The results from the comparison between PSO and GWO showed that both algorithms were capable of producing designs satisfying the equivalence theorem conditions in a large proportion of runs. However, several consistent differences in performance can be observed across the different scenarios.

For the Michaelis-Menten model, PSO consistently outperformed GWO. Across all cases, the runtime distributions for PSO were shifted toward significantly

lower values, with both the median runtime and the interquartile range being noticeably smaller than those of GWO, indicating faster convergence and faster computations on average.

A similar pattern was observed for the stopping iterations n_{stop} . In general, the results for PSO show much fewer iterations with little variability, whereas the results for GWO show a wide range of considerably large numbers of iterations needed to converge.

Moreover, the values of $\Phi(x_i, \xi^*)$ were typically much closer to zero for PSO, and the interquartile ranges were narrow and around very small deviations. In addition, PSO achieved higher success rates across all Michaelis-Menten cases. These results indicate that PSO more frequently produced designs lying closer to the theoretical locally D-optimal solution.

For the Emax model, both algorithms' performances appeared to be substantially worse. Compared to the Michaelis-Menten cases, both PSO and GWO generally required more iterations, had larger variability in runtime and produced larger deviations from the equivalence theorem conditions. This is reasonable given the increased complexity of the Emax model.

Despite the increased difficulty, PSO still consistently achieved shorter runtimes and lower stopping iterations than GWO, which reached the maximum number of iterations many times, indicating difficulties in satisfying the criterion for convergence.

One particularly notable result was observed in Emax Case 2, where GWO performed substantially worse than PSO across multiple diagnostics. Apart from that case, the differences between the algorithms were quite small for the equivalence theorem diagnostics, where both methods often produced comparable deviations from zero. Nevertheless, in general, PSO still achieved slightly higher success rates.

Overall, the results suggest that PSO is more computationally efficient and numerically stable for constructing locally D-optimal designs in the considered nonlinear models. Although GWO was also capable of producing high-quality designs, its convergence was generally slower and less consistent, particularly for the three-parameter Emax scenarios.

6.3 Response Simulation and Parameter Simulation

The simulation study resulted in the locally D-optimal designs generally improving the precision of parameter estimation for the Michaelis-Menten model. Across the considered sample sizes and parameter settings, the parameter estimates obtained under the D-optimal designs typically provided smaller variability than those obtained under the non-optimal designs.

This was clearly visible in the boxplots, where the D-optimal designs produced narrower boxes and shorter whiskers in all cases, indicating smaller interquartile ranges and reduced overall variability of the estimates. Similar patterns were observed in the histograms, where the distributions corresponding to the D-optimal designs were more concentrated and sharply peaked around the true parameter values.

In addition, the estimated means aligned closely with the true parameter values, indicating little bias in the estimators. Overall, these results indicate that the D-optimal designs provided more informative observations for estimation of the Michaelis-Menten parameters.

The results for the three-parameter Emax model showed more complications. In particular, for the sample size $n = 48$, the D-optimal design occasionally produced a few extreme estimates for the parameters V and K . These outliers had a huge influence on the diagnostics and were clearly visible in both the boxplots and histograms. This suggests that estimation under the D-optimal design for the Emax model is more sensitive to noise and numerical instability than in the Michaelis-Menten case.

However, the behavior of the D-optimal designs improved noticeably as the sample size increased. Larger values of n led to fewer and less extreme estimates, far more reasonable diagnostics, and narrower spreads of the distributions, which became increasingly concentrated around the true parameter values.

This behavior is consistent with the theoretical discussion in Section 3.1.3 regarding the Fisher information and the asymptotic properties of the MLE.

It is when the sample size increases that the MLE asymptotically follows a normal distribution with covariance matrix approximated by the inverse of the FIM, leading to $\det(\mathcal{I}(\xi^*; \boldsymbol{\theta}))^{\frac{1}{2}}$ being inversely proportional to the volume of the confidence ellipsoid and D-optimal designs minimizing the uncertainty of the estimates.

Although the D-optimal designs performed well for larger sample sizes, the differences between the D-optimal and non-optimal designs were less pronounced in the Emax case. Both designs often produced relatively similar estimation behavior. One possible explanation is that the considered non-optimal design and the D-optimal design are quite similar in structure. As a result, the non-optimal design may still capture a lot of information.

Taken together, the simulation study supports the theoretical motivation for D-optimality, particularly for the Michaelis-Menten model and for larger sample sizes. At the same time, the results also illustrate that the advantages of using D-optimal designs in practice may depend on factors such as the complexity of the model and the sample size.

6.4 Conclusion

Particle Swarm Optimization (PSO) and the Grey Wolf Optimizer (GWO) produced designs with clear and interpretable structure across the considered choices of parameters.

The support points were consistently located in informative regions of the design space and the equivalence theorem diagnostics showed agreement with the theoretical conditions for local D-optimality, indicating that the meta-heuristics were capable of producing designs that closely satisfy the General Equivalence Theorem.

The comparative analysis between PSO and GWO showed that both algorithms were effective for solving the considered optimization problems, but that PSO generally demonstrated superior performance. In most cases, PSO converged faster, required fewer iterations and achieved higher success rates while producing designs with smaller deviations from the equivalence theorem conditions, and the differences between the algorithms became more

pronounced for the more complex Emax model.

Further, the parameter estimation study demonstrated the practicality of a D-optimal design. In general, for the Michaelis-Menten model, the D-optimal designs produced parameter estimates with smaller variability and tighter concentration around the true parameter values compared to the arbitrarily chosen designs.

For the Emax model, the results were more sensitive to sample size and numerical instability, particularly for smaller samples. However, the performance of the D-optimal designs improved considerably as the sample size increased, which agrees with the underlying asymptotic theory concerning Fisher information and maximum likelihood estimation.

Overall, the results suggest that metaheuristic optimization algorithms, particularly PSO, are effective in constructing locally D-optimal designs for non-linear regression models where analytical solutions are difficult or unavailable. The study remains, however, open for future research and further analysis in a potentially extended framework.

Chapter 7

Appendix

7.1 Code Source Repository

The entire Python implementation for this thesis, including all of the codes that have been considered, is publicly available at:

GitHub repository for the thesis code

The repository contains all of the codes used and explained in Chapter 4 - Approach, that is, codes for:

- computation of locally D-optimal designs,
- verification using the General Equivalence Theorem,
- comparison of PSO and GWO,
- response simulation and parameter estimation.

Bibliography

- Atkinson, Anthony C., Alexander N. Donev, and Randall D. Tobias (2007). *Optimum Experimental Designs, with SAS*. Oxford: Oxford University Press. ISBN: 978-0-19-929659-0.
- Chen, Ping-Yang, Ray-Bing Chen, Heng-Chin Tung, et al. (2017). “Standardized maximin D-optimal designs for enzyme kinetic inhibition models”. In: *Chemometrics and Intelligent Laboratory Systems* 169, pp. 79–86. DOI: 10.1016/j.chemolab.2017.08.009.
- Chen, Ping-Yang, Ray-Bing Chen, and Weng Kee Wong (2022). “Particle swarm optimization for searching efficient experimental designs: A review”. In: *WIREs Computational Statistics*, e1578. DOI: 10.1002/wics.1578.
- Fedorov, Valerii V. and Peter Hackl (2025). *Model-Oriented Design of Experiments*. 2nd ed. Vol. 196. Lecture Notes in Statistics. Springer. ISBN: 978-1-0716-4301-3. DOI: 10.1007/978-1-0716-4302-0.
- Goos, Peter and Bradley Jones (2011). *Optimal Design of Experiments: A Case Study Approach*. Chichester, West Sussex: John Wiley & Sons, Ltd. ISBN: 978-0-470-74461-1.
- Johnson, Richard A. and Dean W. Wichern (2014). *Applied Multivariate Statistical Analysis*. 6th ed. Harlow, Essex: Pearson Education Limited. ISBN: 978-1-292-02494-3.
- Keating, Kim A. and James F. Quinn (1998). “Estimating species richness: The Michaelis-Menten model revisited”. In: *Oikos* 81.2, pp. 411–416.
- Kiefer, Jack (1974). “General Equivalence Theory for Optimum Designs (Approximate Theory)”. In: *The Annals of Statistics* 2.5, pp. 849–879.
- Liero, Hannelore and Silvelyn Zwanzig (2006). *Introduction to the Theory of Statistical Inference*. Springer.
- López-Fidalgo, Jesús (2023). *Optimal Experimental Design: A Concise Introduction for Researchers*. Vol. 226. Lecture Notes in Statistics. Cham:

- Springer Nature Switzerland. ISBN: 978-3-031-35917-0. DOI: 10.1007/978-3-031-35918-7.
- Luke, Sean (2009). *Essentials of Metaheuristics*. Available online at <http://cs.gmu.edu/~sean/book/metaheuristics/>. Lulu.
- Magnusdottir, Bergrun T. (2016). “Optimal designs for a multiresponse Emax model and efficient parameter estimation”. In: *Biometrical Journal* 58.3, pp. 518–534. DOI: 10.1002/bimj.201400203.
- Mirjalili, Seyedali, Seyed Mohammad Mirjalili, and Andrew Lewis (2014). “Grey Wolf Optimizer”. In: *Advances in Engineering Software* 69, pp. 46–61. DOI: 10.1016/j.advengsoft.2013.12.007.
- Navarro, Danielle (Nov. 11, 2024). *Bayesian Estimation for Emax Regression*. URL: https://blog.djnavarro.net/posts/2024-11-11_emax_parameters/.
- Tarek, Mohamed, Jose Storopoli, and Joel Owen (2024). *Introduction to Optimal Design*. `IntroductiontoOptimalDesigntutorial`. Tutorial document.